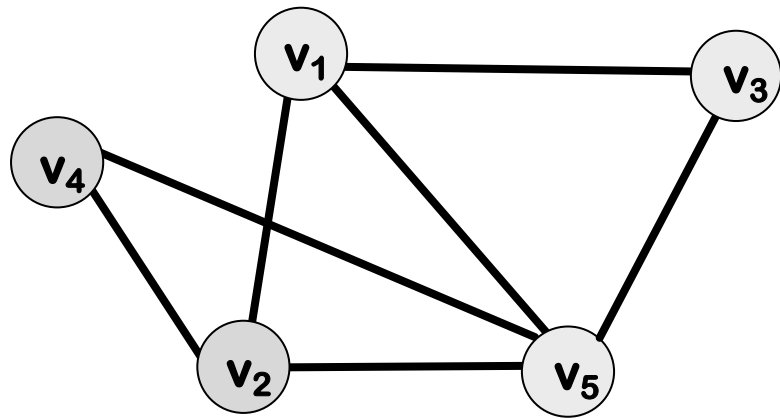




GRAPH TRAVERSAL TECHNIQUES

MARZIEH ESKANDARI
NJIT

WHAT IS A GRAPH?

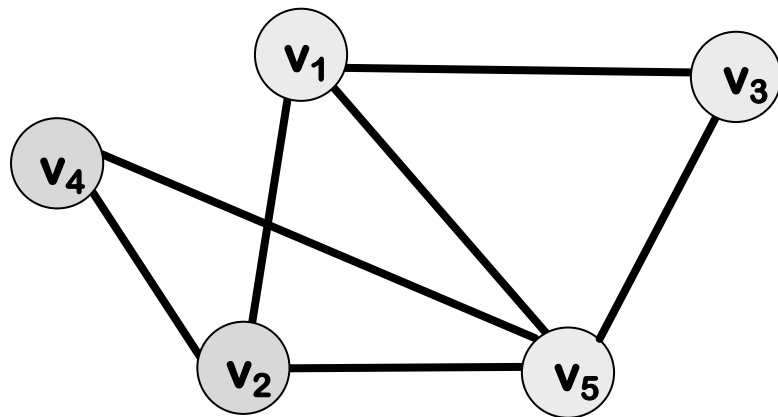


$$G=(E,V)$$

$$V = \{v_1, v_2, v_3, v_4, v_5\}$$

$$E = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_1, v_5\}, \\ \{v_2, v_5\}, \{v_2, v_4\}, \{v_3, v_5\}, \{v_4, v_5\}\}$$

WHAT IS A GRAPH?

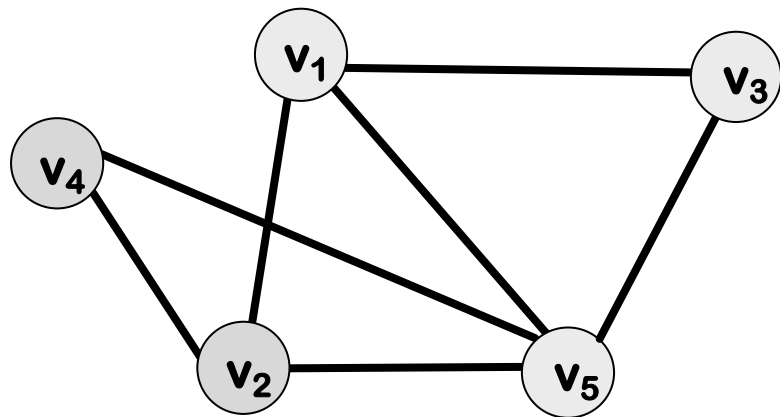


$$G=(E,V)$$

Number of nodes: $|V|$

Number of edges: $|E|$

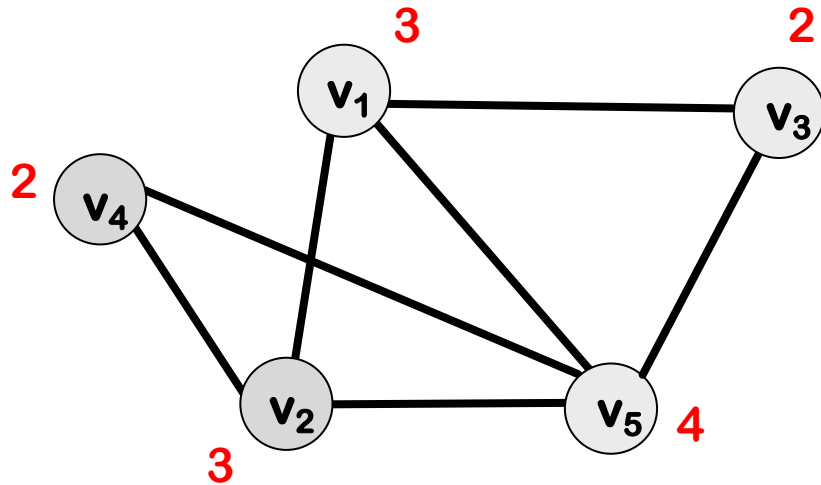
WHAT IS A GRAPH?



$$G=(E,V)$$

$$|V| = n$$

DEGREE IN AN UNDIRECTED GRAPH



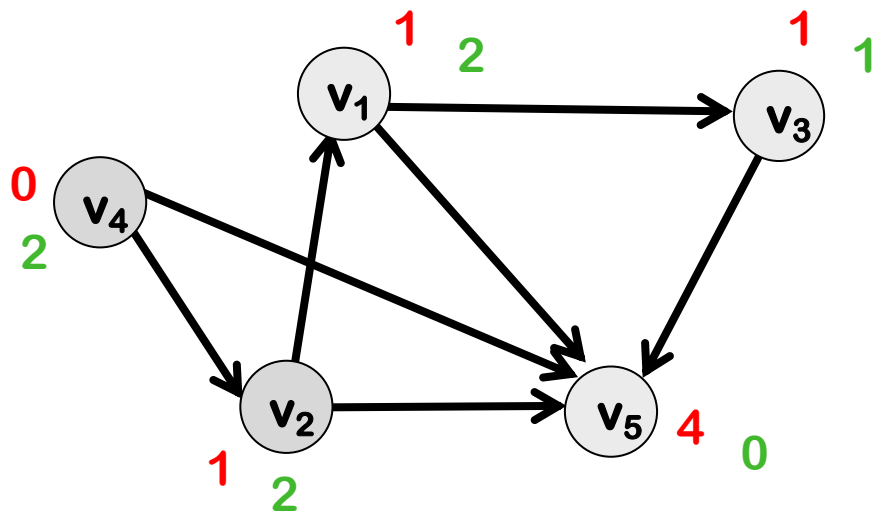
$G=(E,V)$

Degree of a node, $\deg(v_i)$:
the number of edges connected to it.

$$\deg(v_1) + \deg(v_2) + \cdots + \deg(v_n) = 2 \times |E|$$

$$3 + 3 + 2 + 2 + 4 = 2 \times 7$$

DEGREE IN A DIRECTED GRAPH



$$G=(E,V)$$

Indegree of a node:

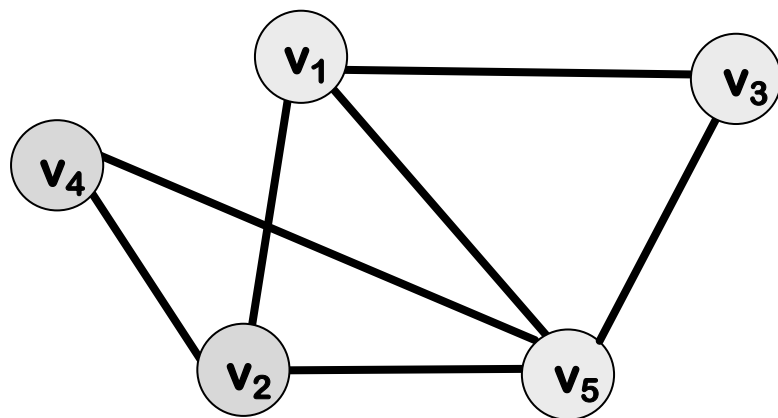
the number of incoming edges

Outdegree of a node:

the number of outgoing edges

ADJACENCY MATRIX REPRESENTATION

Undirected
Unweighted



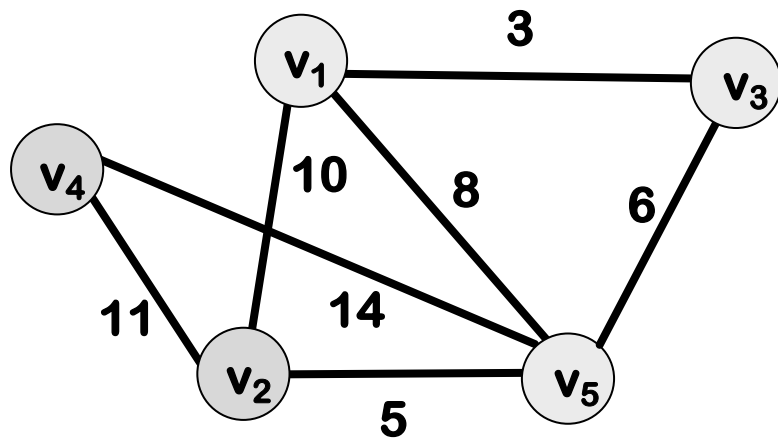
G

	v_1	v_2	v_3	v_4	v_5
v_1	0	1	1	0	1
v_2	1	0	0	1	1
v_3	1	0	0	0	1
v_4	0	1	0	0	1
v_5	1	1	1	1	0

W

ADJACENCY MATRIX REPRESENTATION

Undirected
Weighted



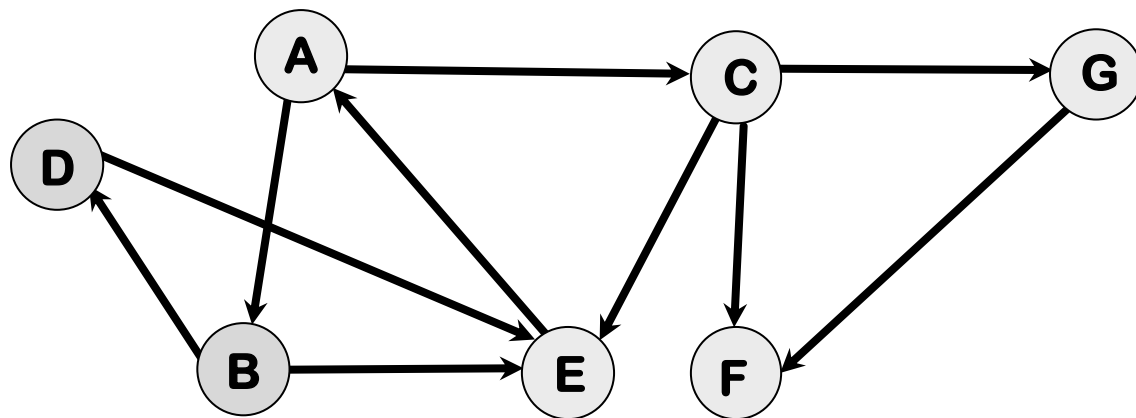
G

	v_1	v_2	v_3	v_4	v_5
v_1	0	10	3	∞	8
v_2	10	0	∞	11	5
v_3	3	∞	0	∞	6
v_4	∞	11	∞	0	14
v_5	8	5	6	14	0

W

ADJACENCY MATRIX

Directed
Unweighted



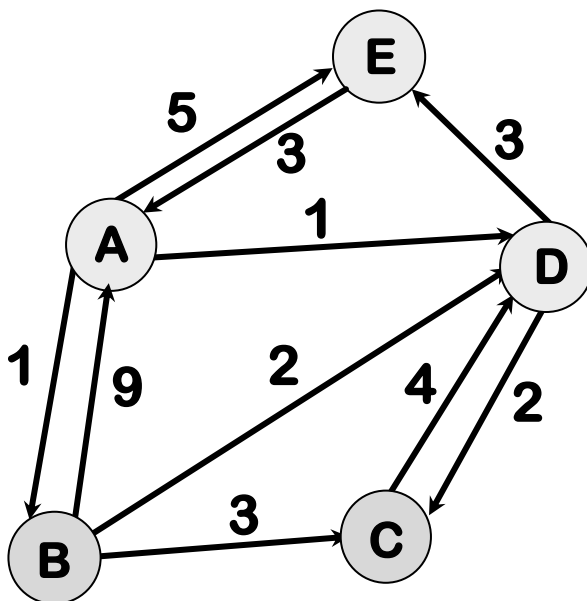
G

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	0	0	0	1	1	0	0
C	0	0	0	0	1	1	1
D	0	0	0	0	1	0	0
E	1	0	0	0	0	0	0
F	0	0	0	0	0	0	0
G	0	0	0	0	0	1	0

W

ADJACENCY MATRIX

Directed
Weighted



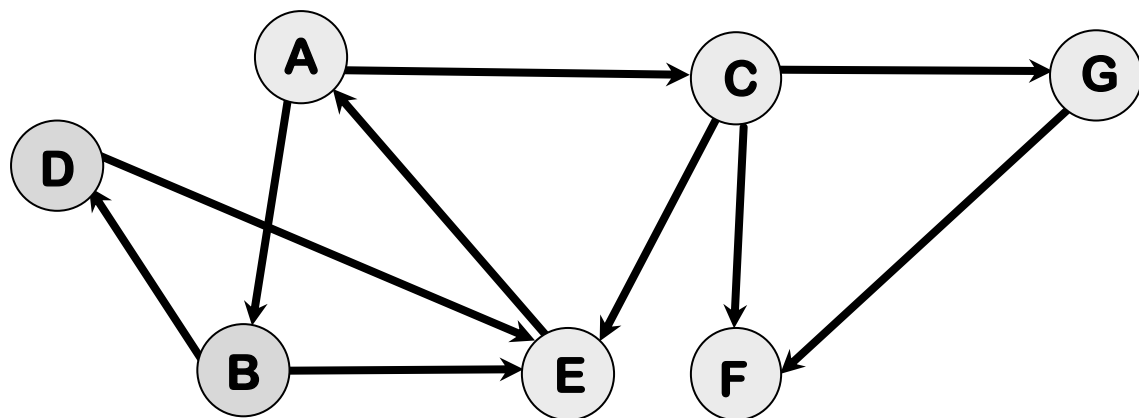
G



	A	B	C	D	E
A	0	1	∞	1	5
B	9	0	3	2	∞
C	∞	∞	0	4	∞
D	∞	∞	2	0	3
E	3	∞	∞	∞	0

W

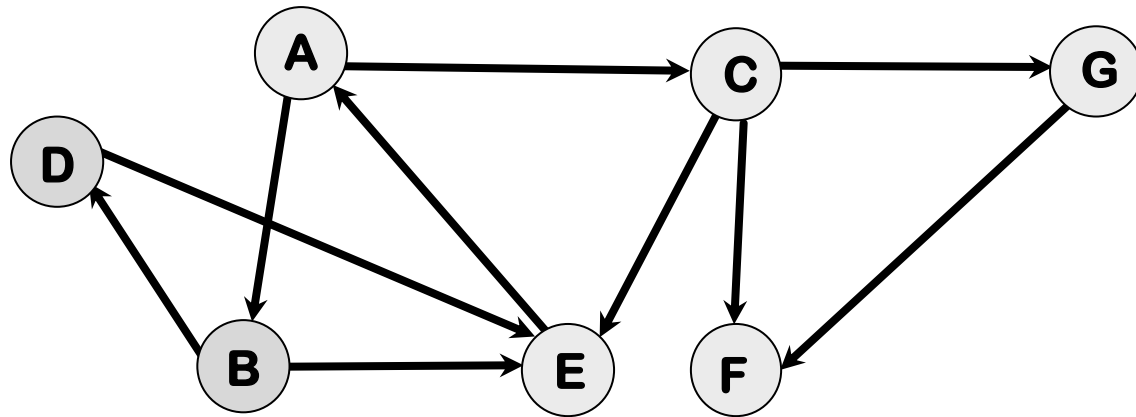
ADJACENCY LIST STRUCTURE



G

NODE	NEIGHBORS
A	B → C
B	D → E
C	E → F → G
D	E
E	A
F	
G	F

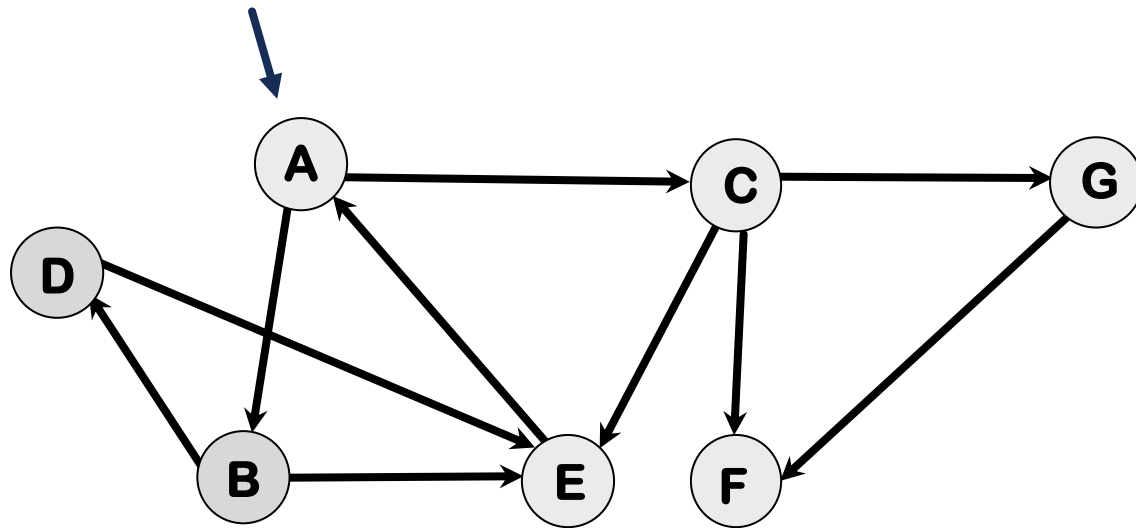
GRAPH: TRAVERSAL



G

NODE	NEIGHBORS
A	B,C
B	D,E
C	E,F,G
D	E
E	A
F	-
G	F

TRAVERSING A GRAPH?



G

NODE	NEIGHBORS	LABEL
A	B,C	X
B	D,E	X
C	E,F,G	X
D	E	X
E	A	X
F	-	X
G	F	X

TRAVERSING A GRAPH?

- **Depth-First Search: DFS**

search deeper whenever possible

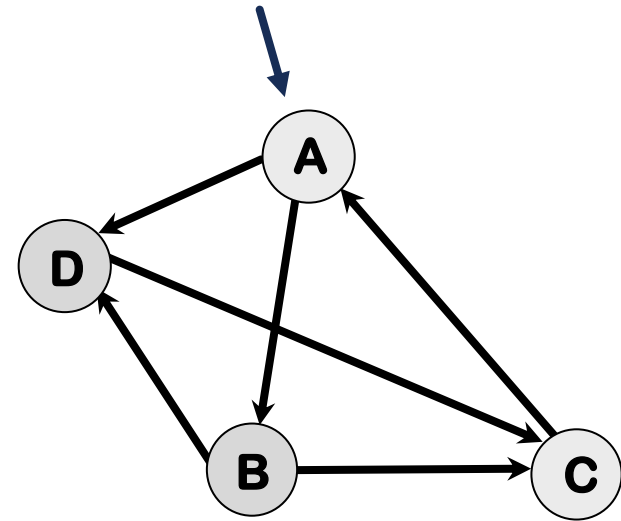
NOT Unique!

- **Breadth-First Search: BFS**

search in breadth first manner whenever possible

ABCD

ADBC



TRAVERSING A GRAPH?

- **Depth-First Search: DFS**

search deeper whenever possible

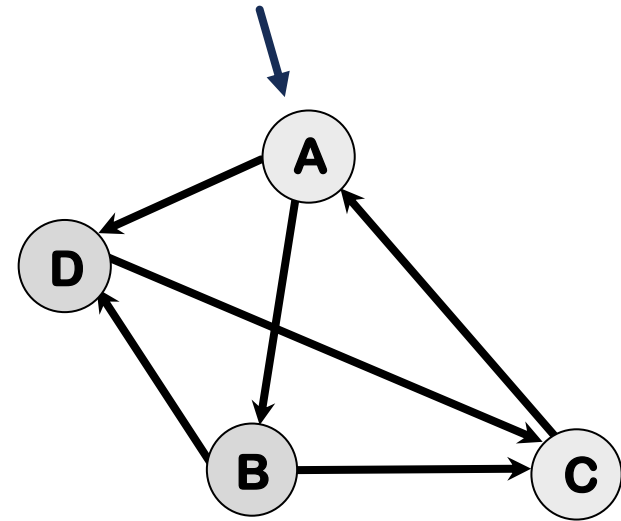
NOT Unique!

- **Breadth-First Search: BFS**

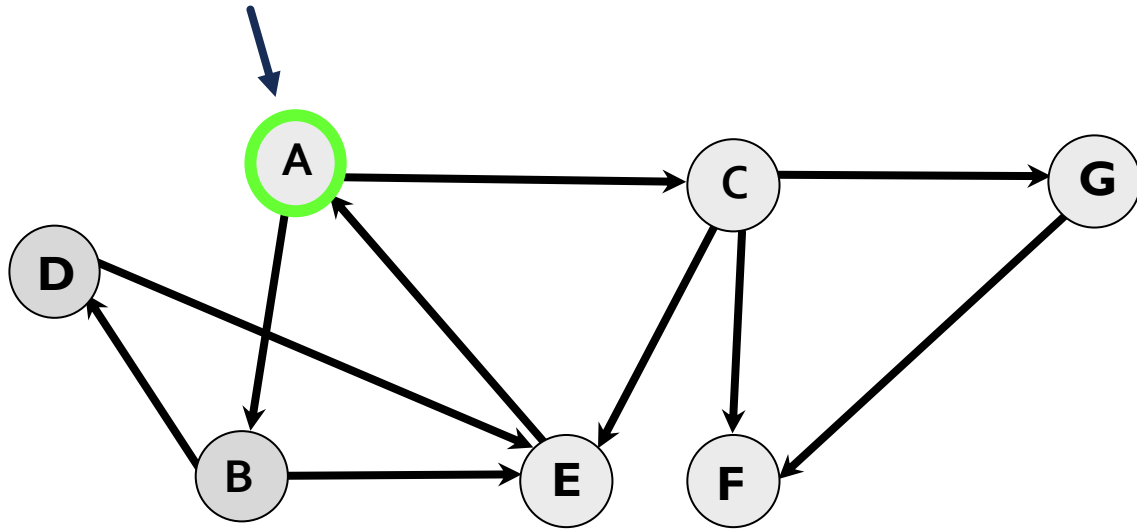
search in breadth first manner whenever possible

ABCD, ABDC, ADCB,

ADBC, ABDC

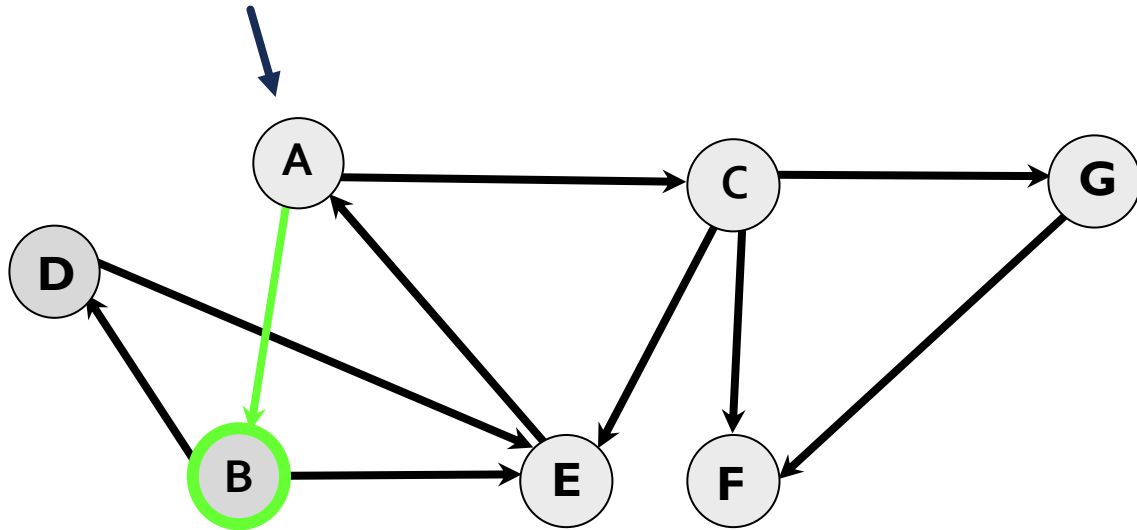


DEPTH-FIRST SEARCH



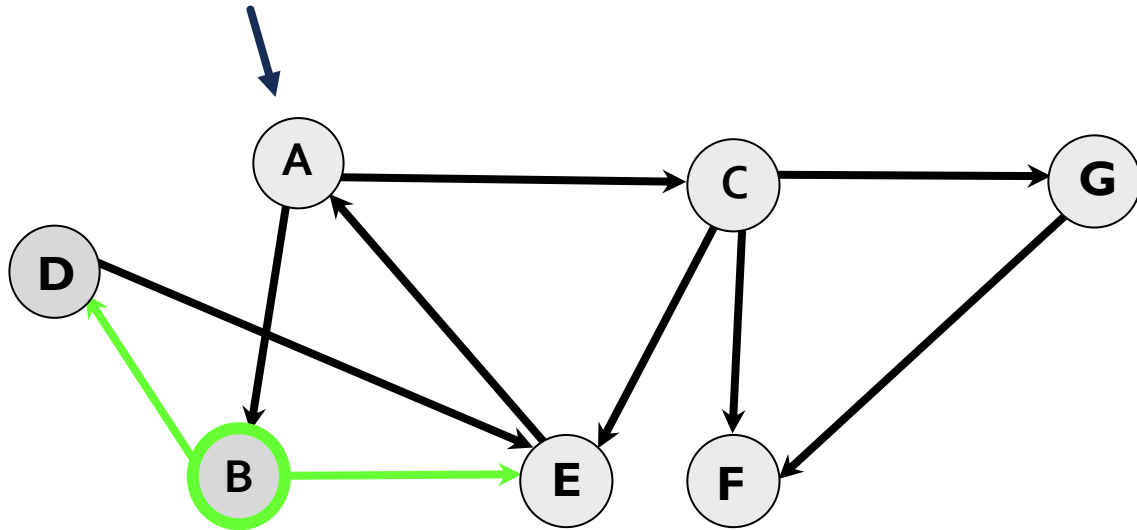
A

DEPTH-FIRST SEARCH



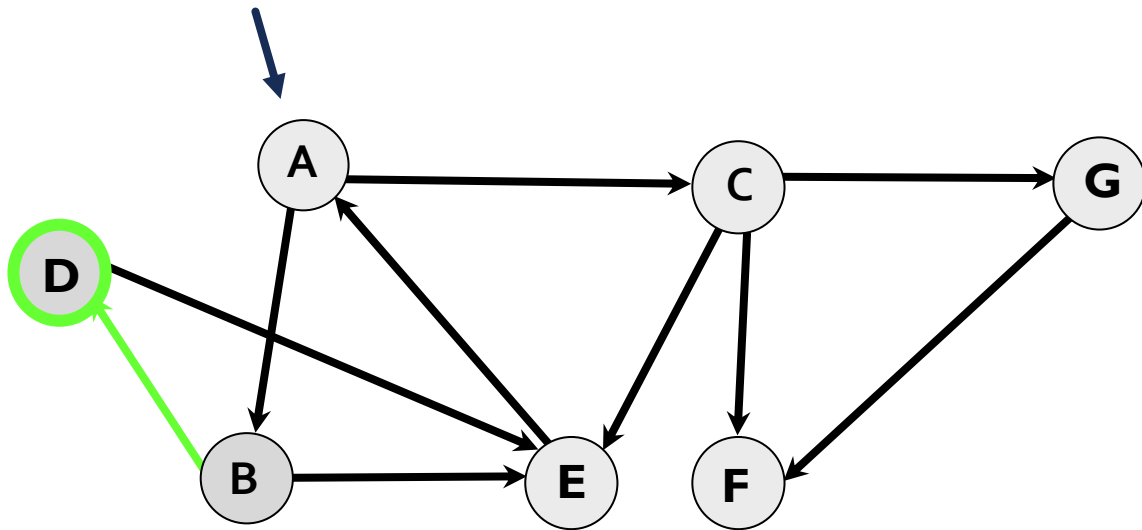
A B

DEPTH-FIRST SEARCH



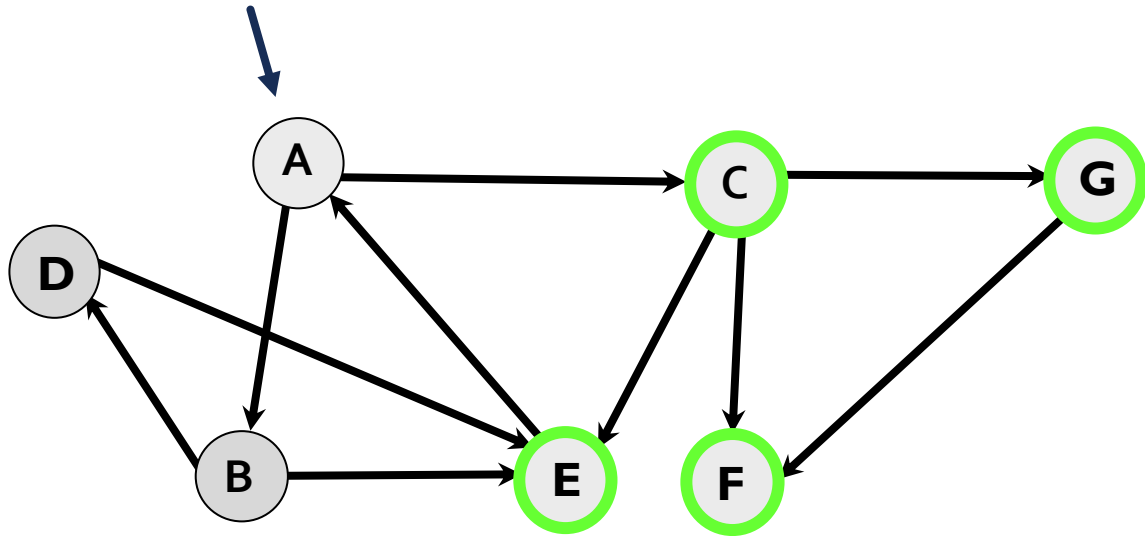
A B

DEPTH-FIRST SEARCH



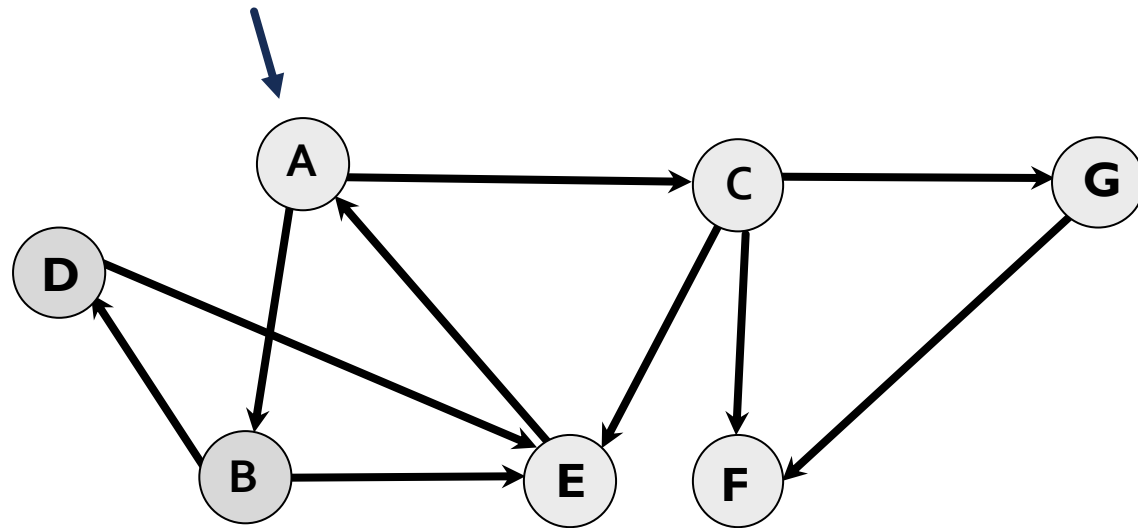
A B D

DEPTH-FIRST SEARCH



A B D E C F G

DEPTH-FIRST SEARCH

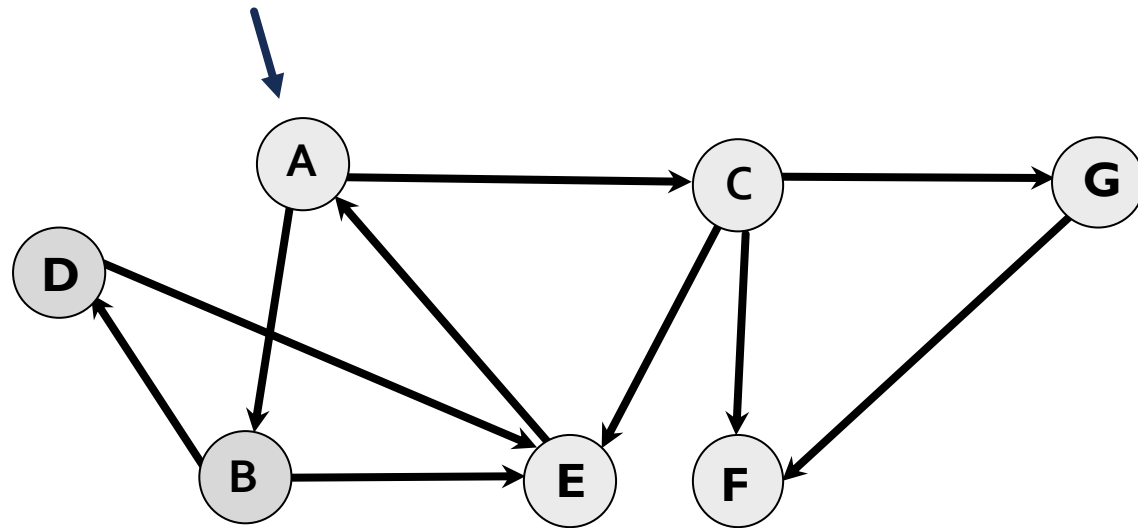


S



NODE	NEIGHBORS	LABEL
A	B,C	X
B	D,E	X
C	E,F,G	X
D	E	X
E	A	X
F	-	X
G	F	X

DEPTH-FIRST SEARCH

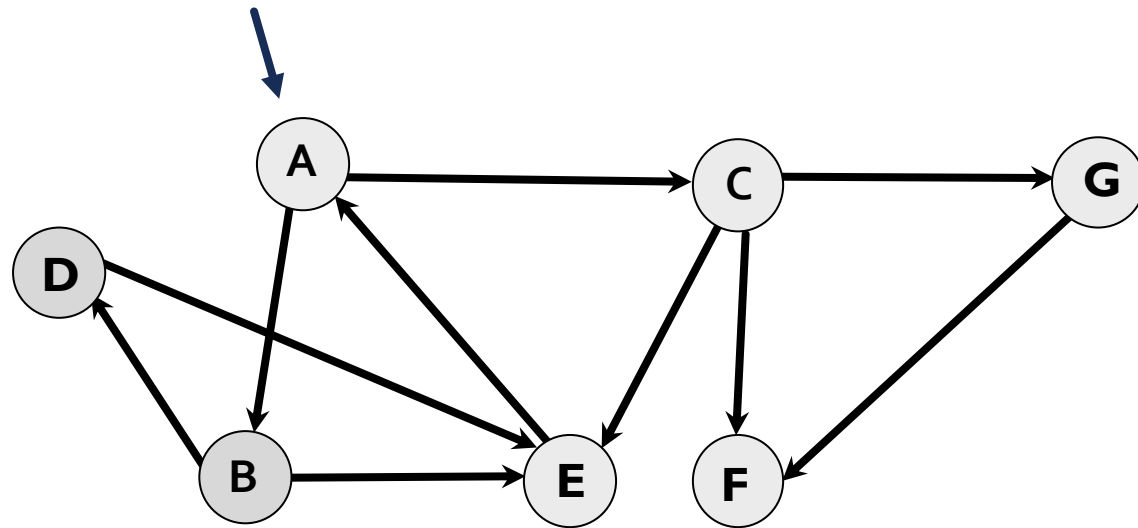


S

A

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	X
C	E,F,G	X
D	E	X
E	A	X
F	-	X
G	F	X

DEPTH-FIRST SEARCH



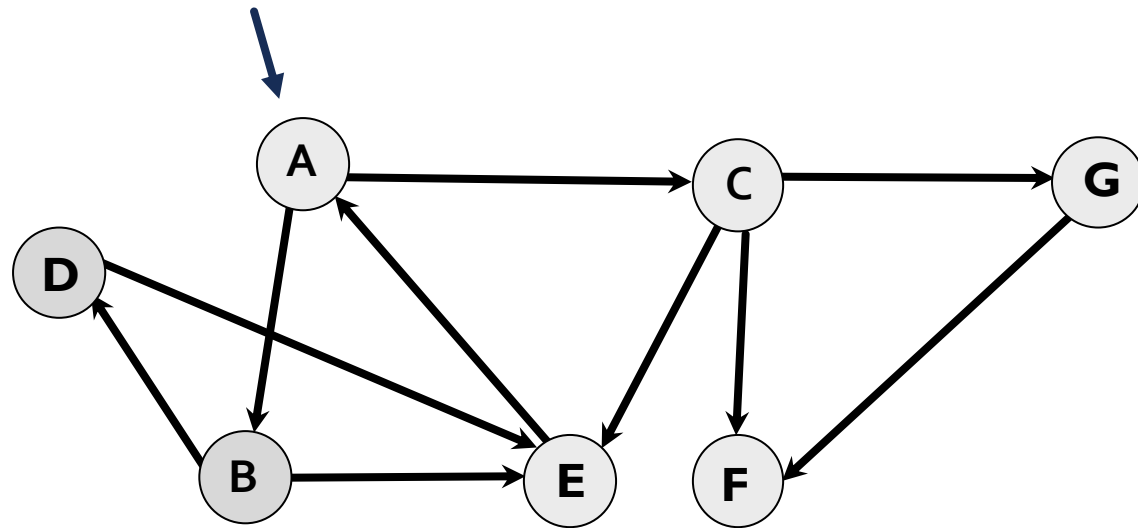
S



A

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	X
C	E,F,G	X
D	E	X
E	A	X
F	-	X
G	F	X

DEPTH-FIRST SEARCH



S

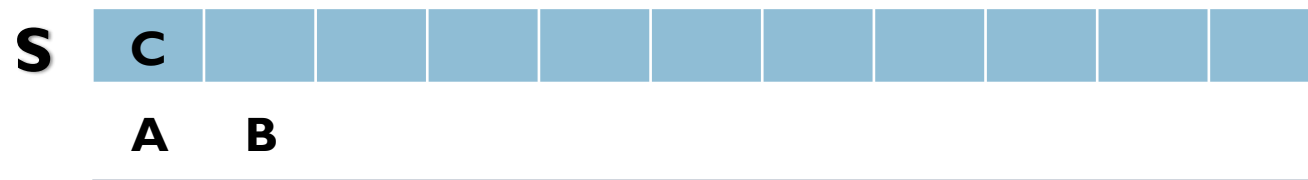
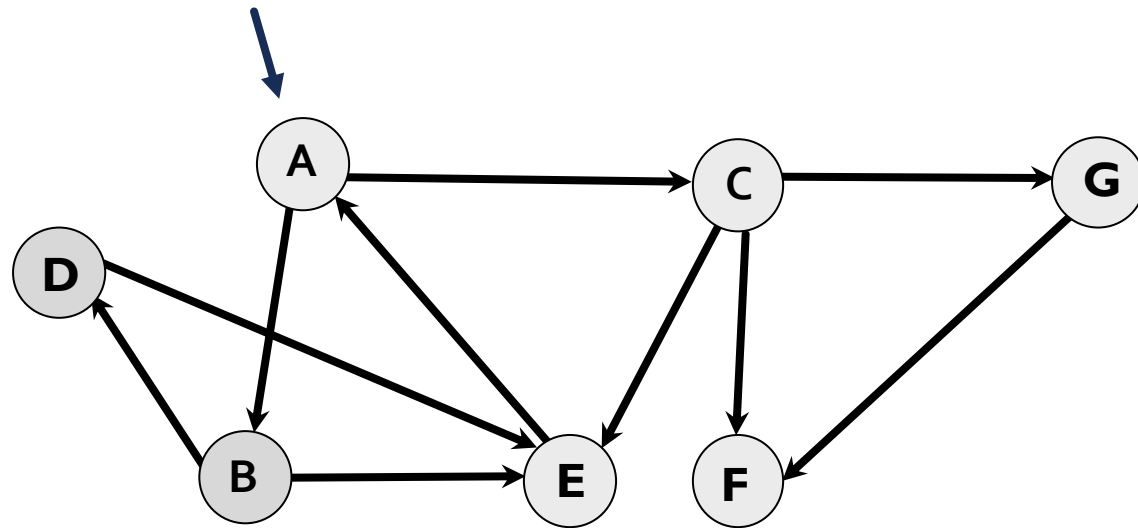
C

B

A

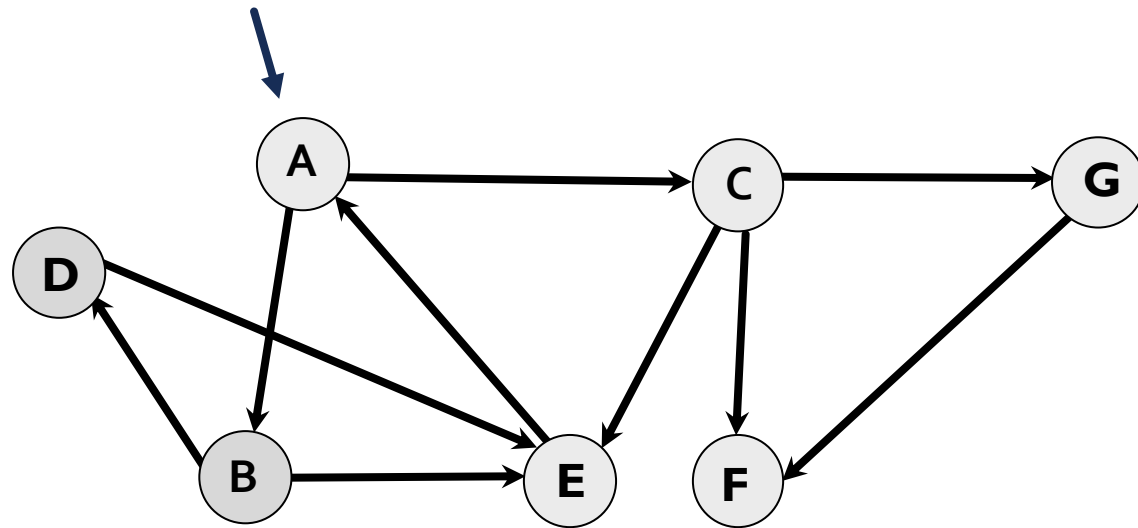
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✗
E	A	✗
F	-	✗
G	F	✗

DEPTH-FIRST SEARCH



NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	X
E	A	X
F	-	X
G	F	X

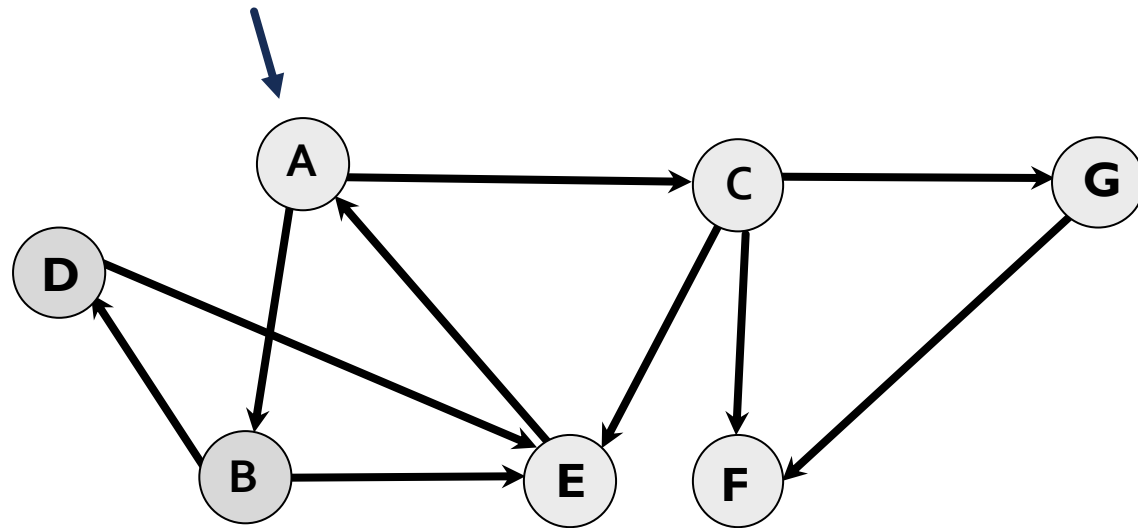
DEPTH-FIRST SEARCH



S	C	E	D							
	A	B								

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✗
G	F	✗

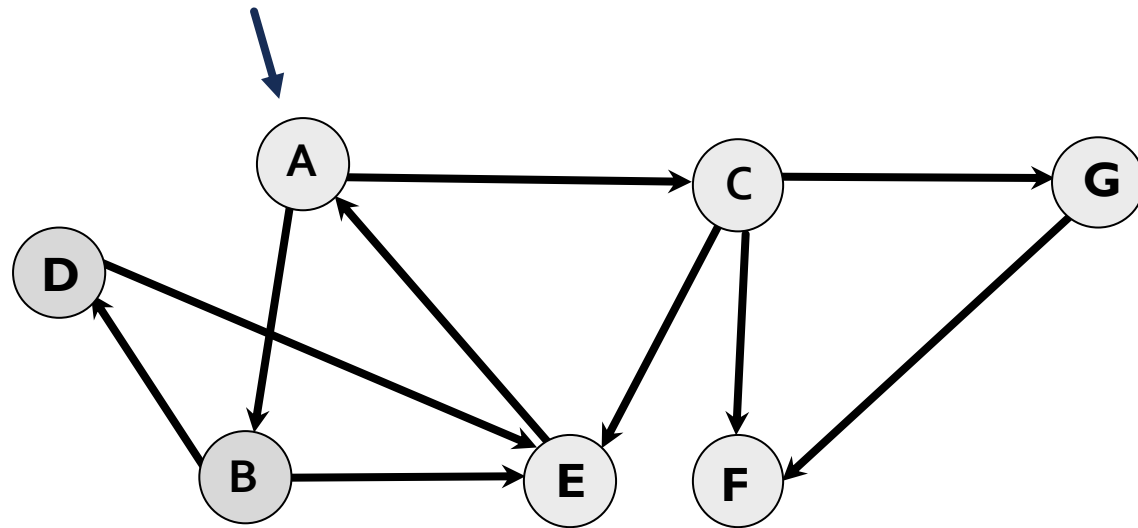
DEPTH-FIRST SEARCH



S	C	E								
	A	B	D							

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✗
G	F	✗

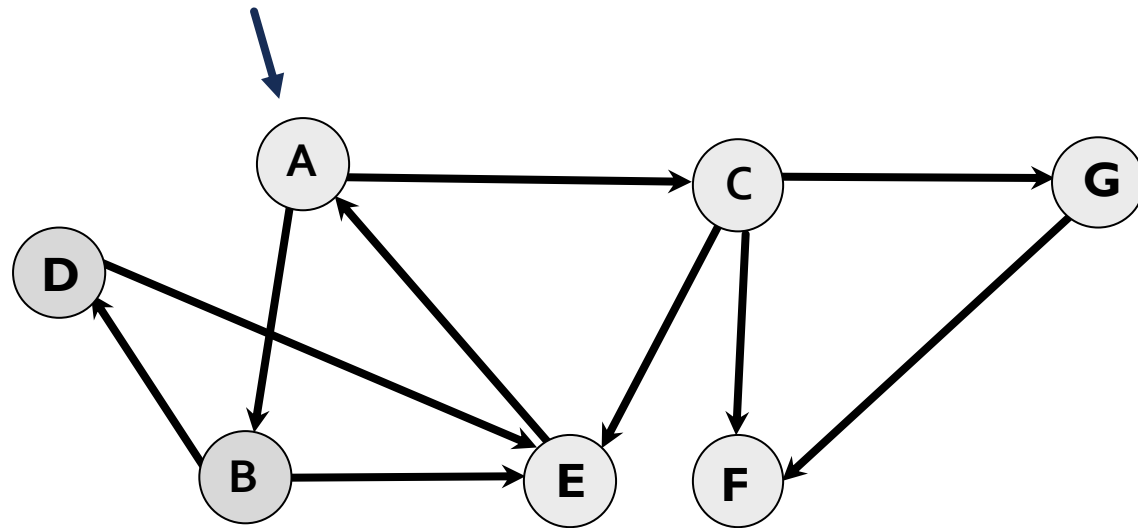
DEPTH-FIRST SEARCH



S	C									
	A	B	D	E						

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✗
G	F	✗

DEPTH-FIRST SEARCH



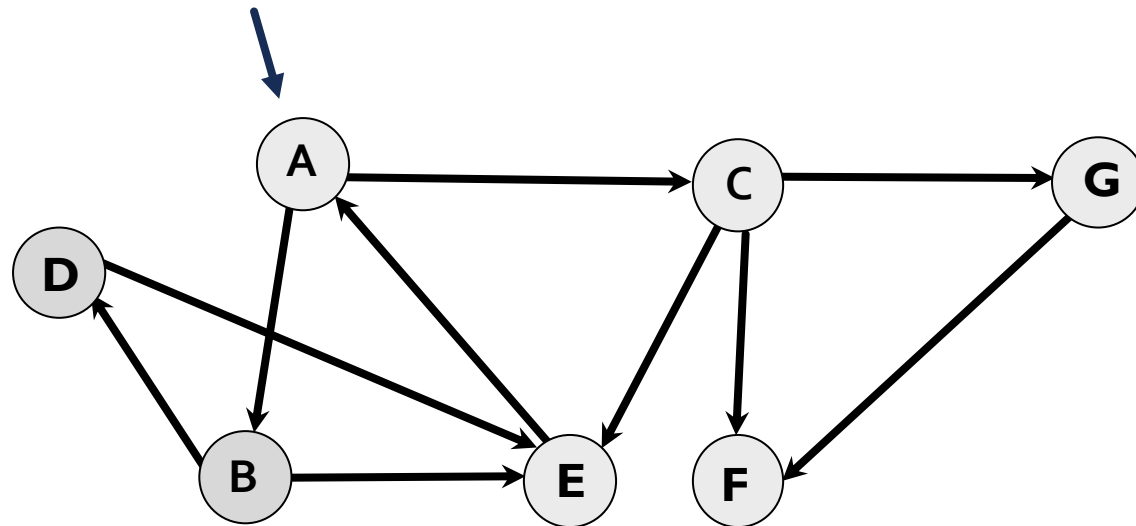
S



A B D E C

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	X
G	F	X

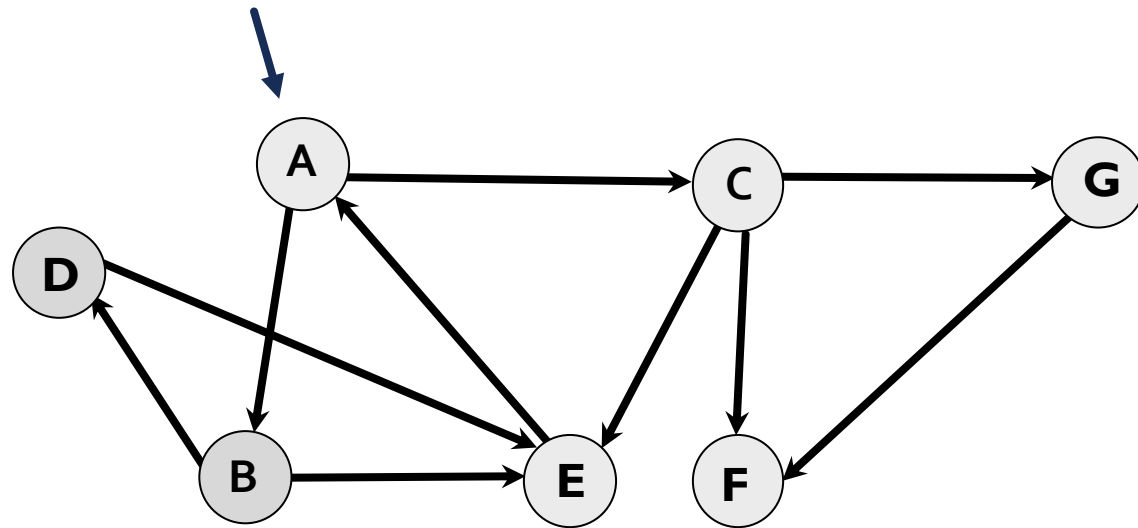
DEPTH-FIRST SEARCH



S	G	F								
	A	B	D	E	C					

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

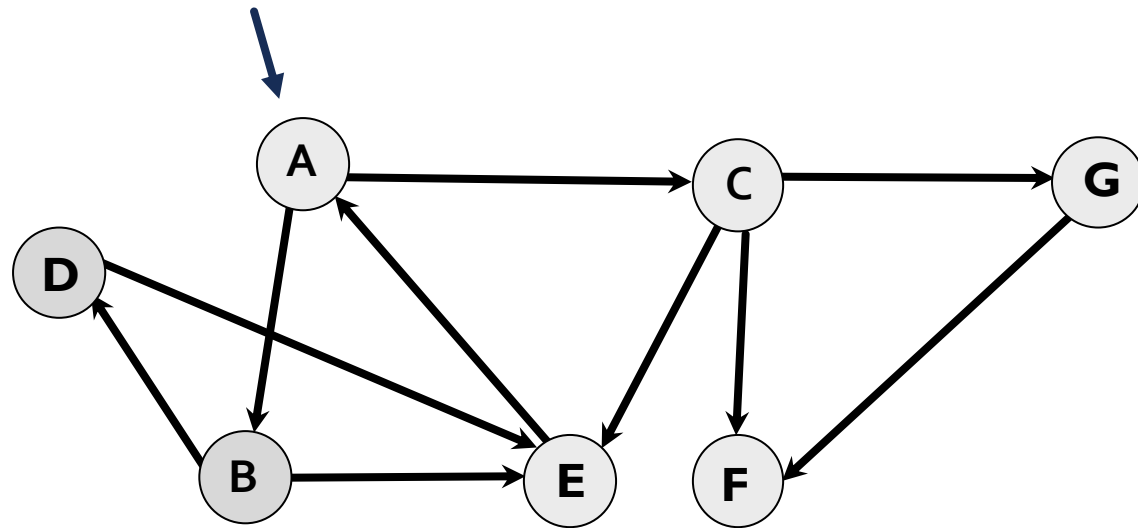
DEPTH-FIRST SEARCH



S	G									
	A	B	D	E	C	F				

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

DEPTH-FIRST SEARCH



S



A B D E C F G

NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

DFS: ALGORITHM AND TIME COMPLEXITY

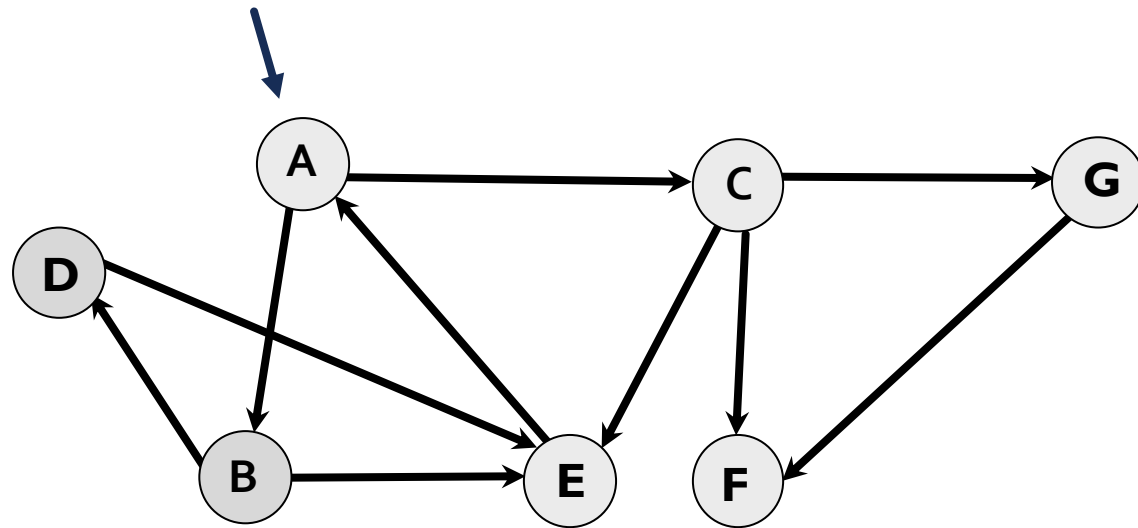
```
DFS(START)
  Stack S
  Mark all nodes as unchecked
  S.push(START)
  Mark START as checked

  while S is not empty do
    v ← S.pop()
    Print v

    for each neighbor w of v do
      if w is unchecked then
        S.push(w)
        Mark w as checked
```

$O(|E| + |V|)$

BREADTH-FIRST SEARCH

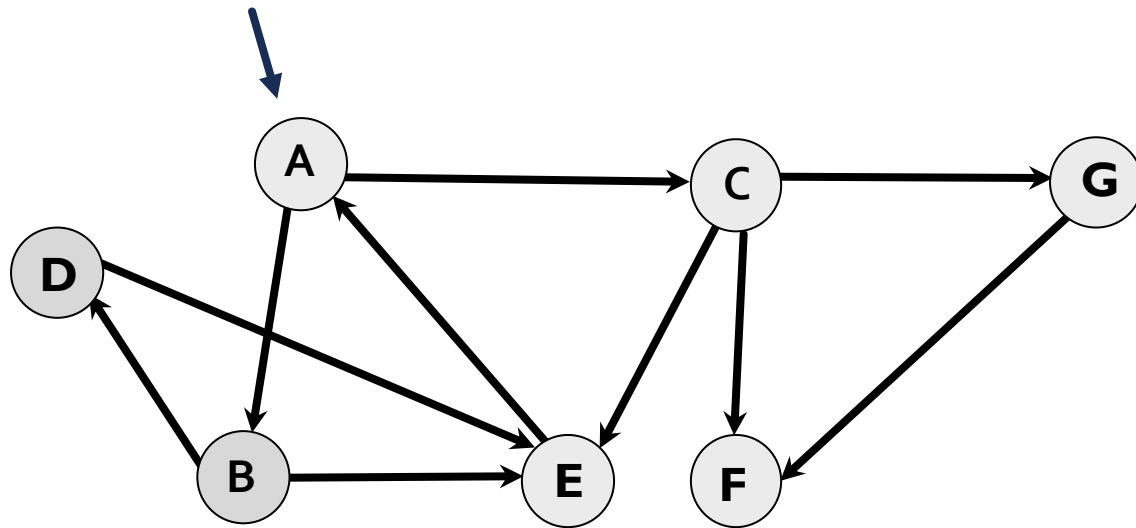


Q



NODE	NEIGHBORS	LABEL
A	B,C	X
B	D,E	X
C	E,F,G	X
D	E	X
E	A	X
F	-	X
G	F	X

BREADTH-FIRST SEARCH

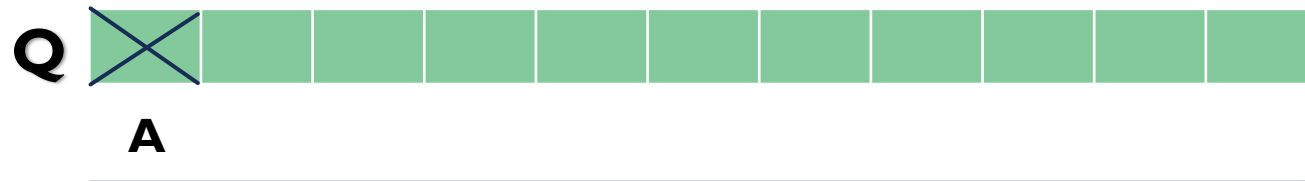
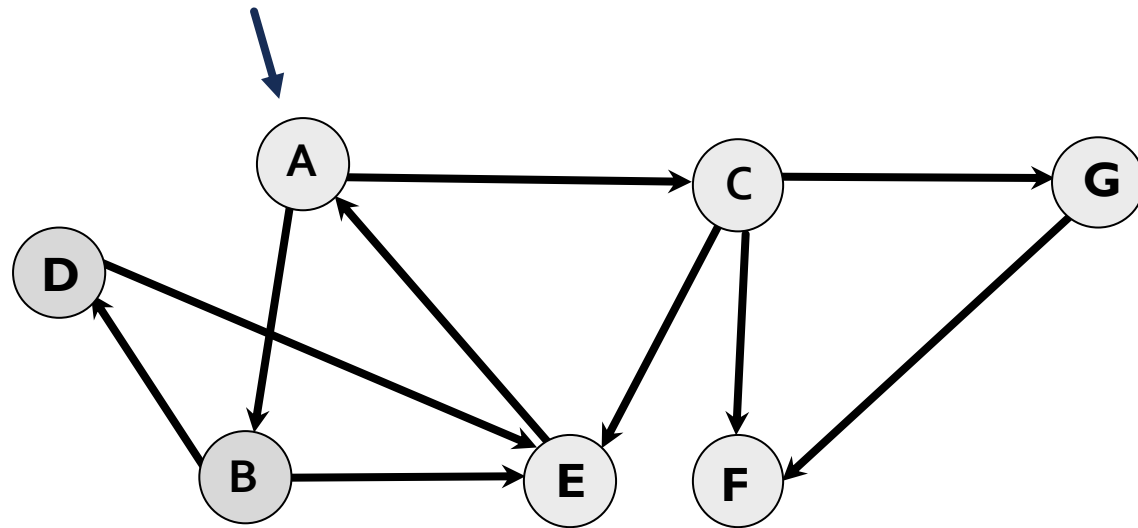


Q

A

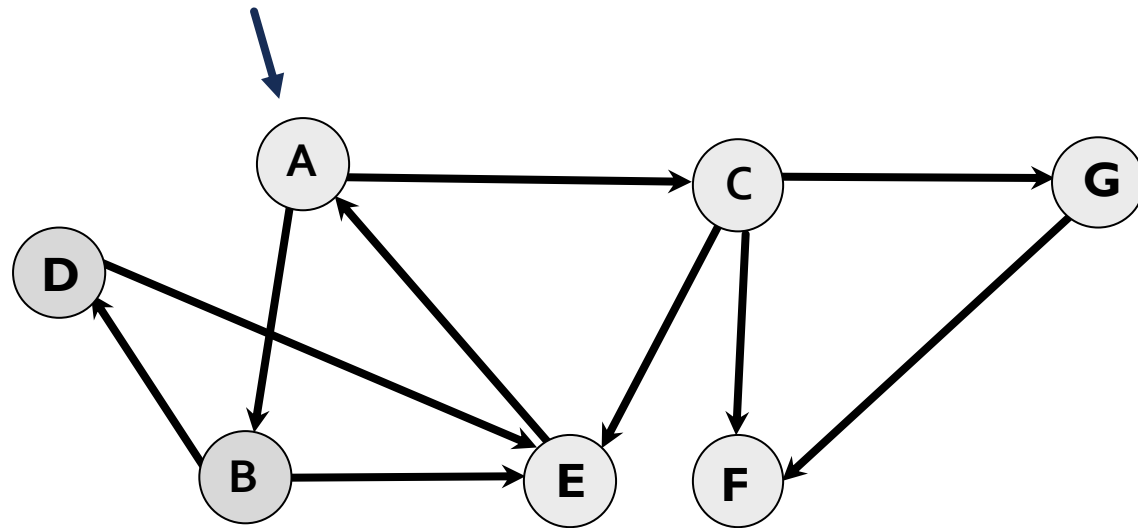
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	X
C	E,F,G	X
D	E	X
E	A	X
F	-	X
G	F	X

BREADTH-FIRST SEARCH



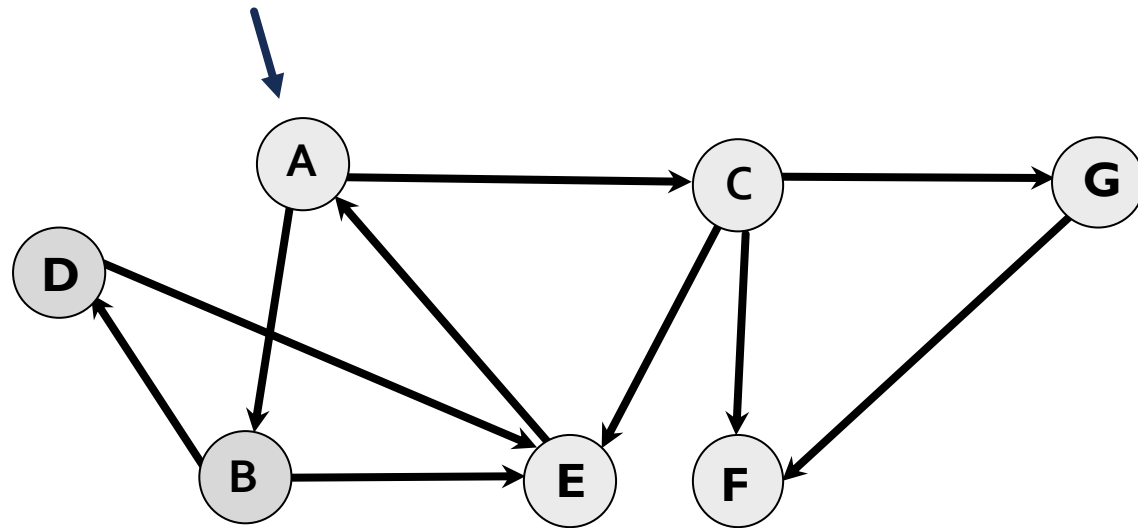
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	X
C	E,F,G	X
D	E	X
E	A	X
F	-	X
G	F	X

BREADTH-FIRST SEARCH



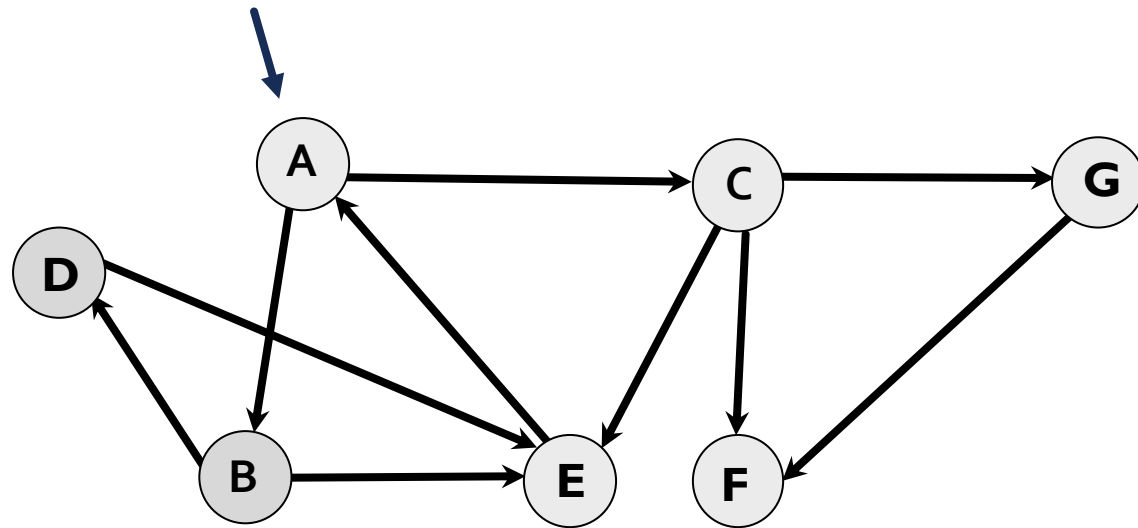
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	X
E	A	X
F	-	X
G	F	X

BREADTH-FIRST SEARCH



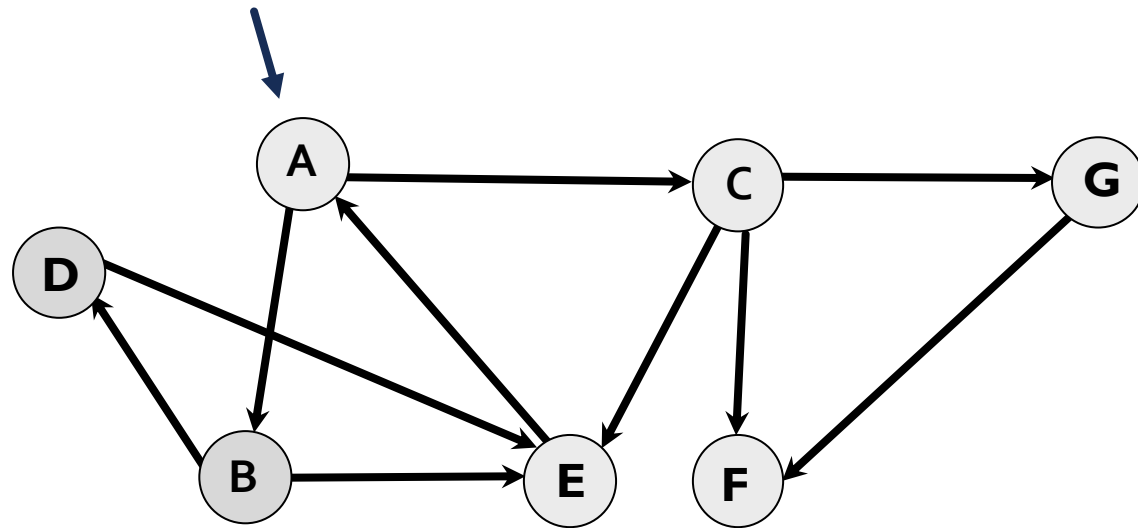
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	X
E	A	X
F	-	X
G	F	X

BREADTH-FIRST SEARCH



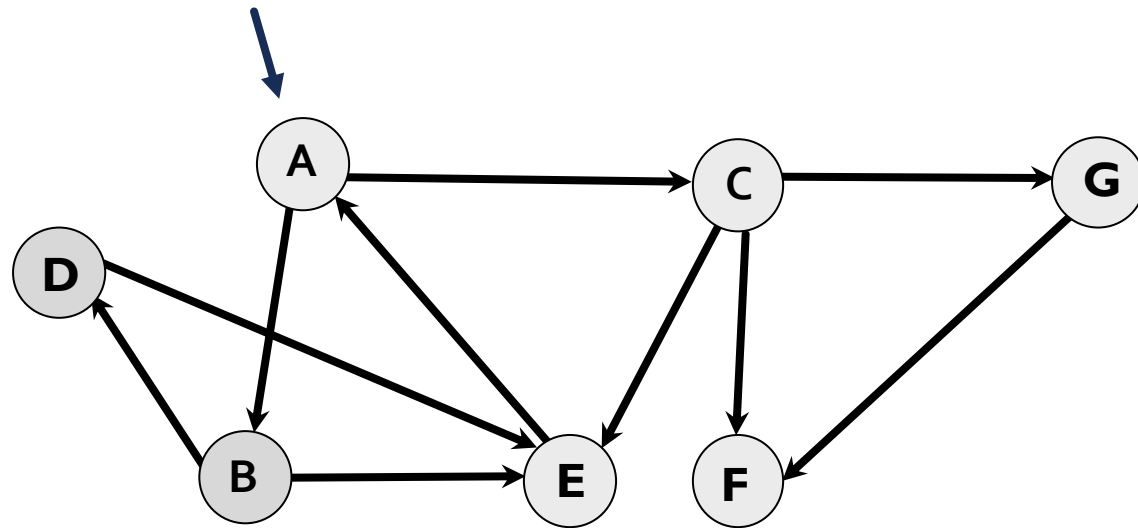
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✗
E	A	✓
F	-	✓
G	F	✓

BREADTH-FIRST SEARCH



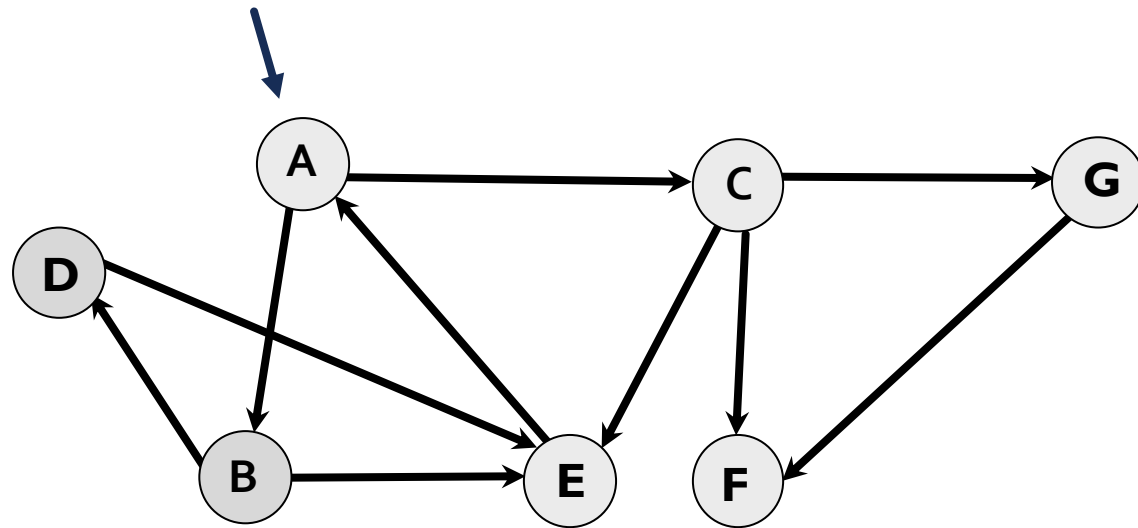
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✗
E	A	✓
F	-	✓
G	F	✓

BREADTH-FIRST SEARCH



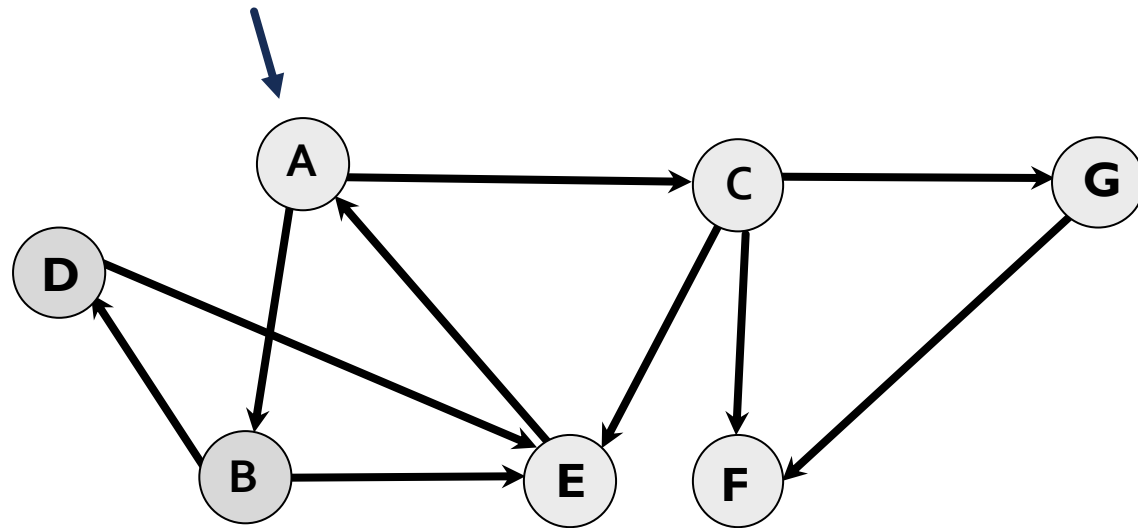
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

BREADTH-FIRST SEARCH



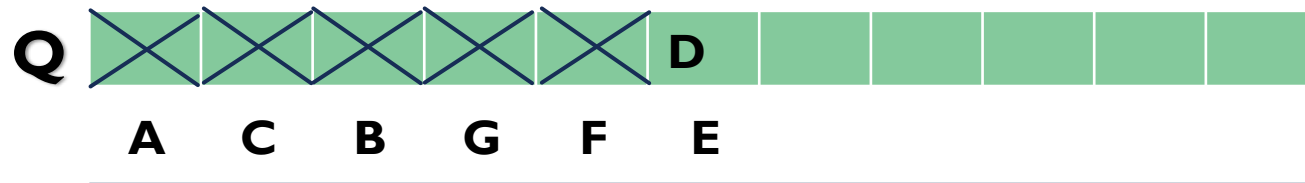
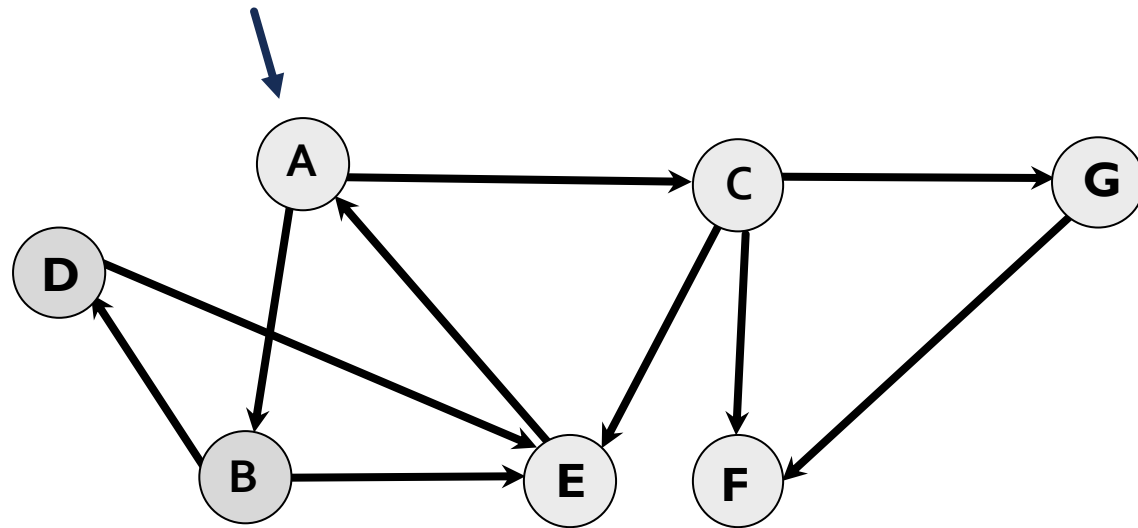
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

BREADTH-FIRST SEARCH



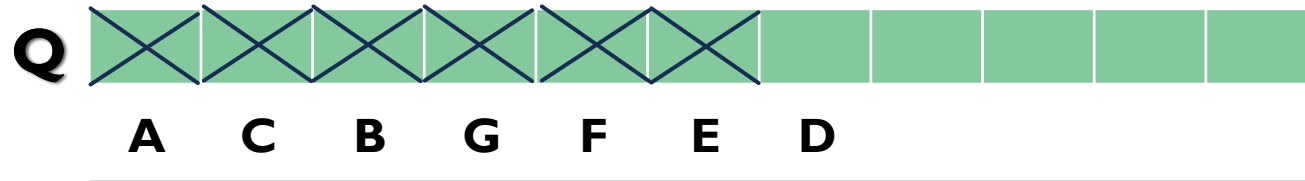
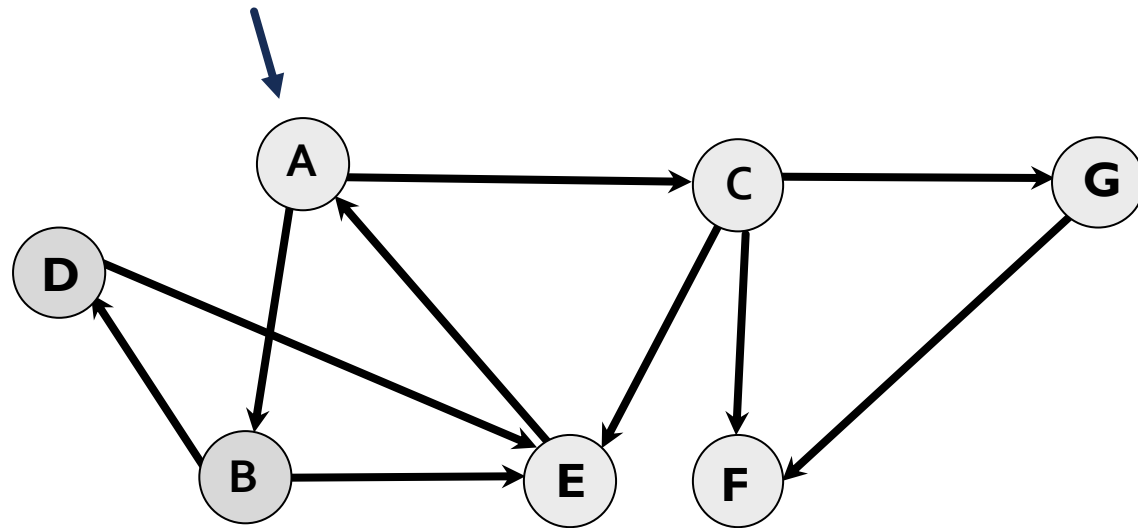
NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

BREADTH-FIRST SEARCH



NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

BREADTH-FIRST SEARCH



NODE	NEIGHBORS	LABEL
A	B,C	✓
B	D,E	✓
C	E,F,G	✓
D	E	✓
E	A	✓
F	-	✓
G	F	✓

BFS:ALGORITHM AND TIME COMPLEXITY

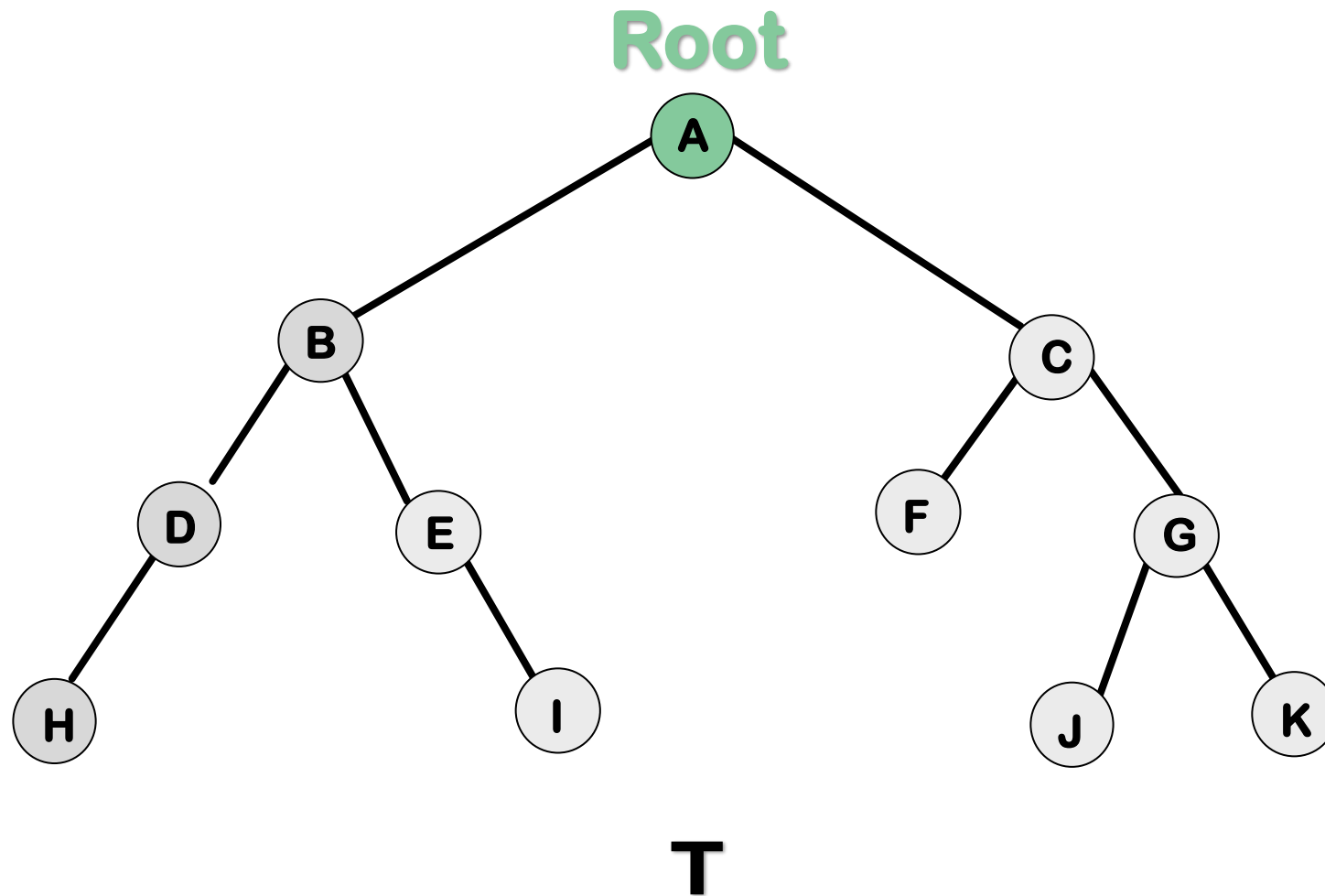
```
BFS(START)
  Queue Q
  Mark all nodes as unchecked
  Q.add(START)
  Mark START as checked

  while Q is not empty do
    v ← Q.delete()
    Print v

    for each neighbor w of v do
      if w is unchecked then
        Q.add(w)
        Mark w as checked
```

$O(|E| + |V|)$

BINARY TREE



TRAVERSING A TREE?

- **Depth-First Search: DFS**

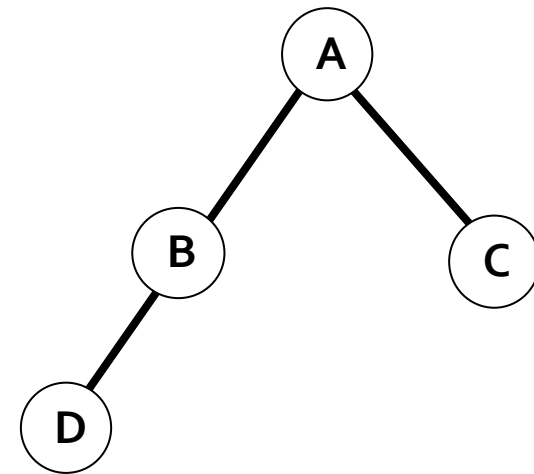
search deeper whenever possible

- **Breadth-First Search: BFS**

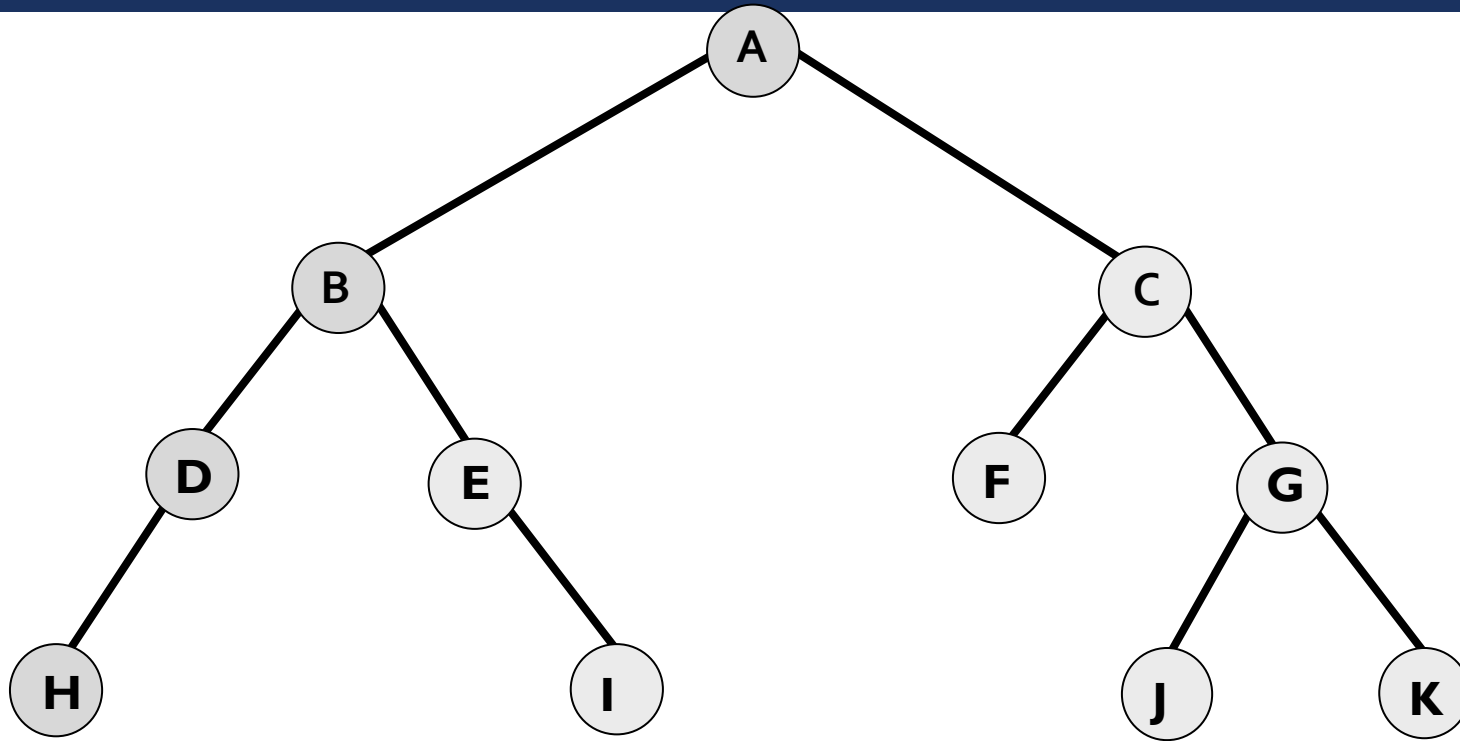
search in breadth first manner whenever possible

ABDC

ABCD



DEPTH-FIRST SEARCH

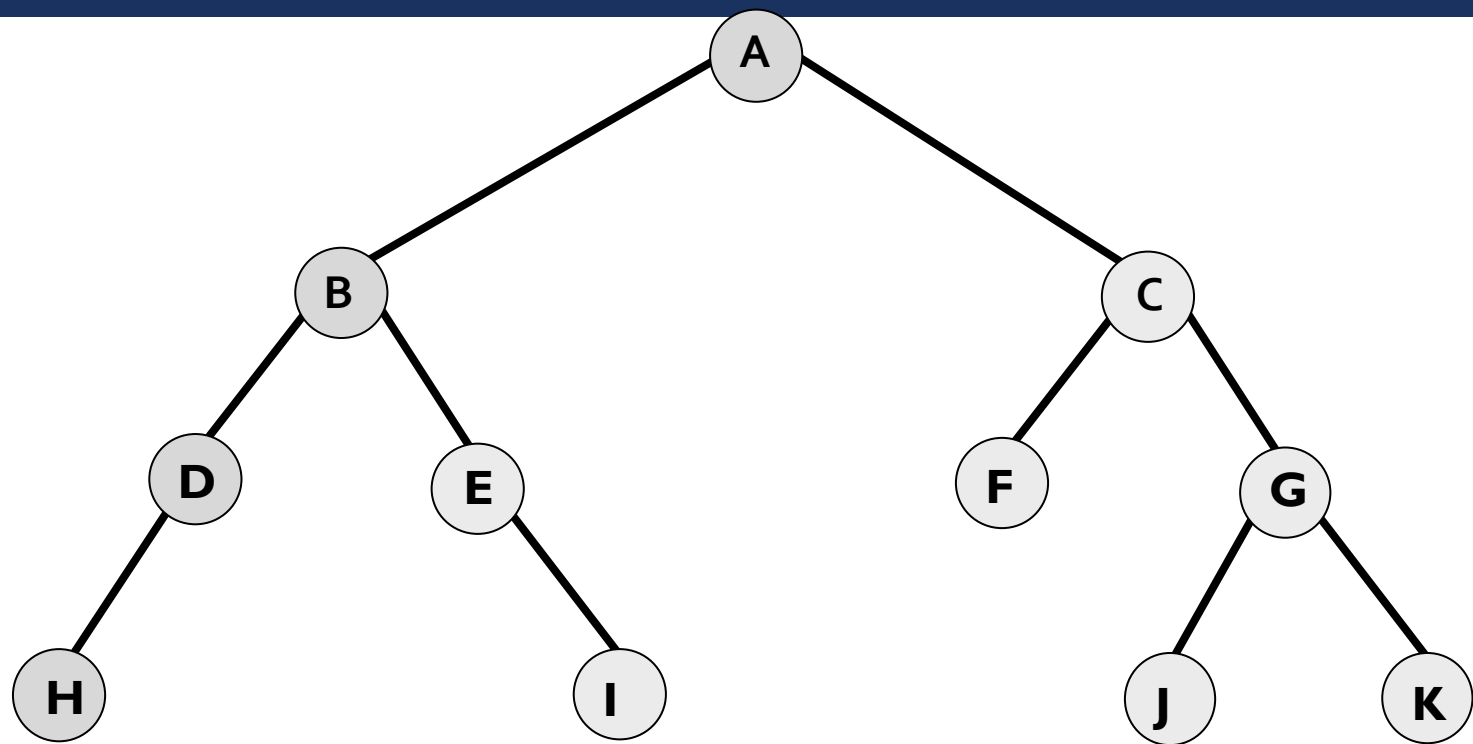


S



NOD E	CHILDRE N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH

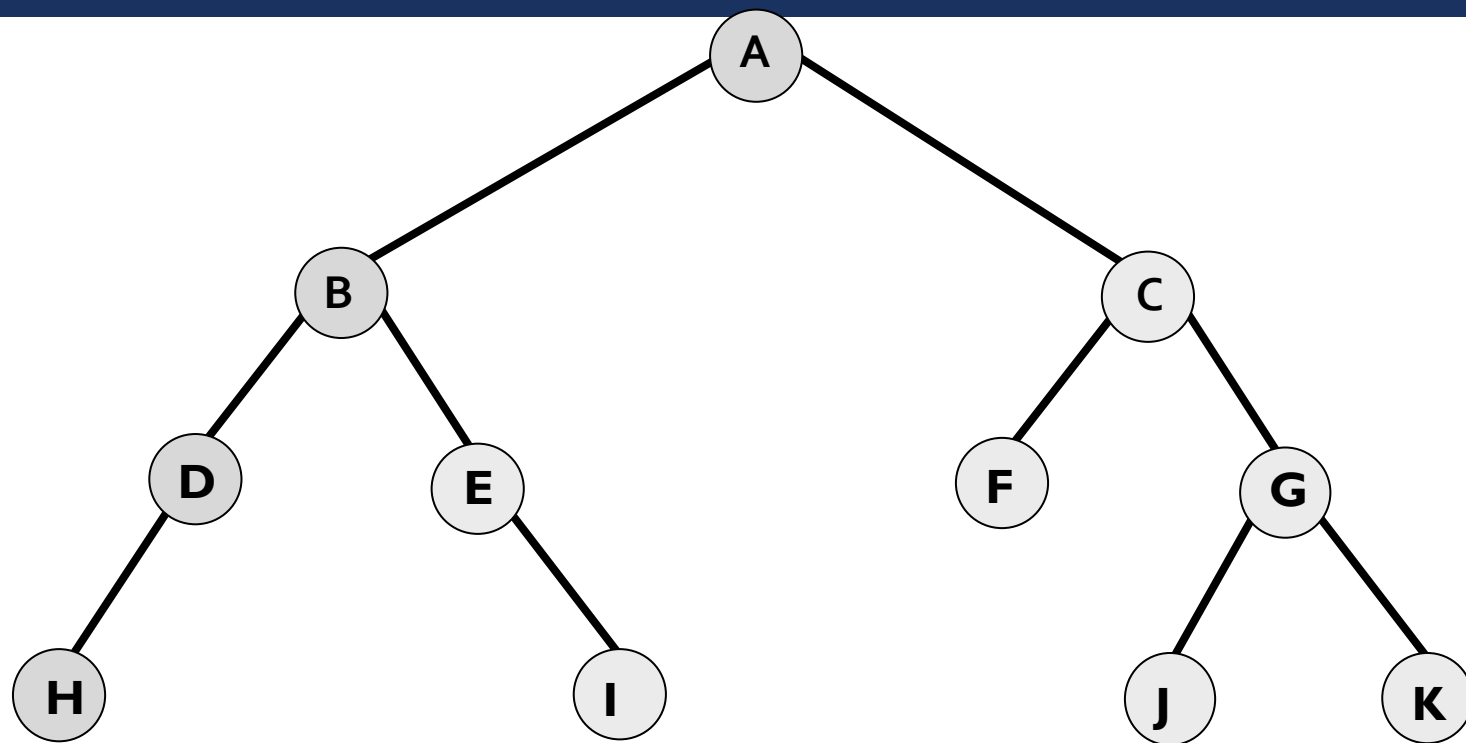


S



NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

DEPTH-FIRST SEARCH



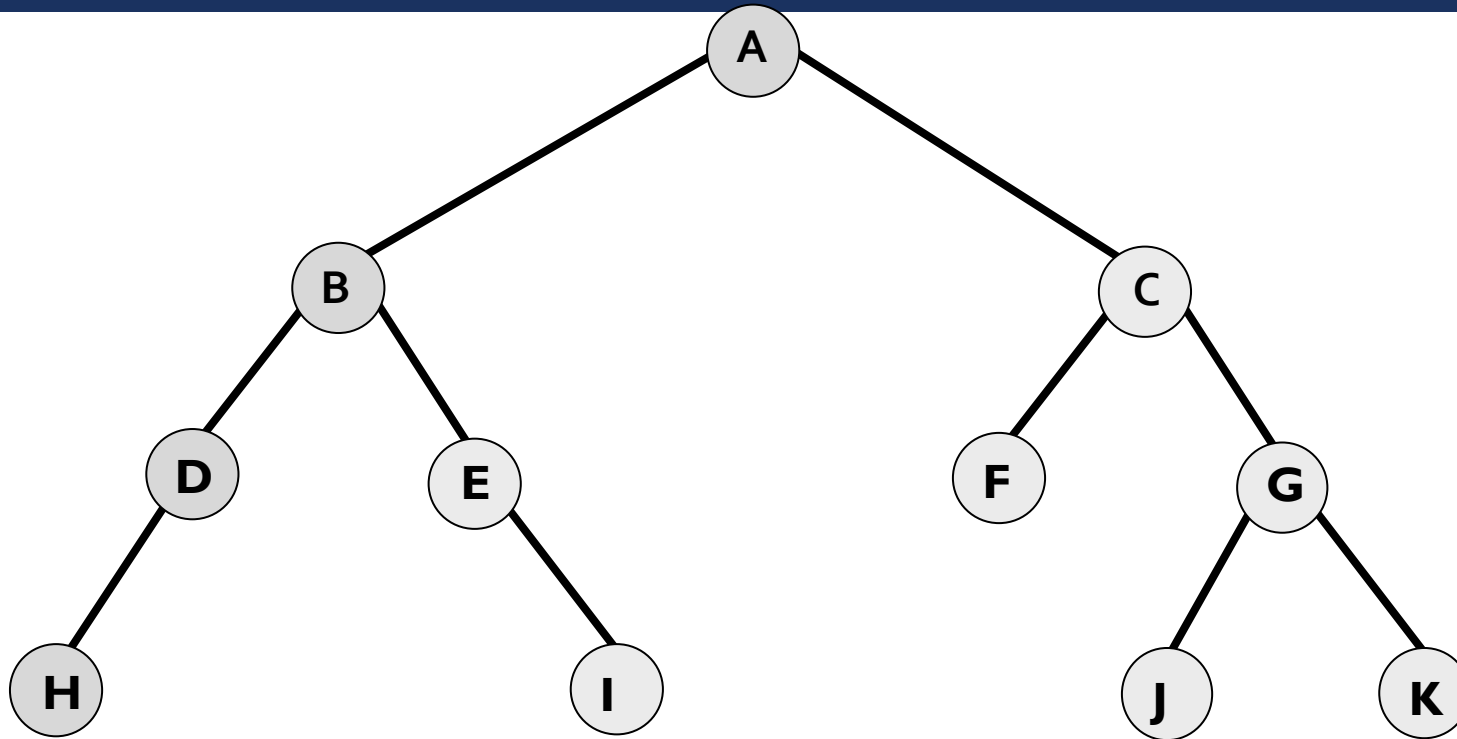
S



A

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH



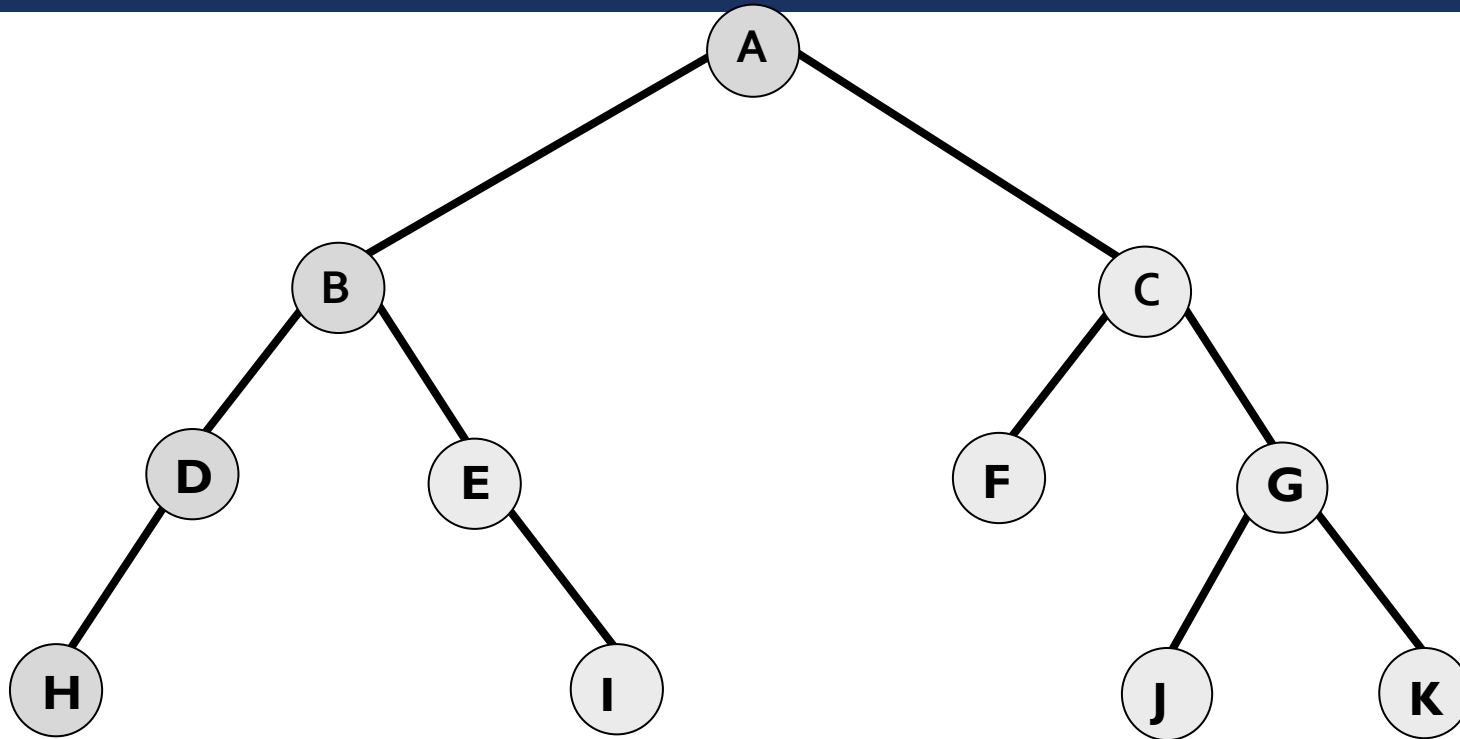
S



A

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

DEPTH-FIRST SEARCH



S

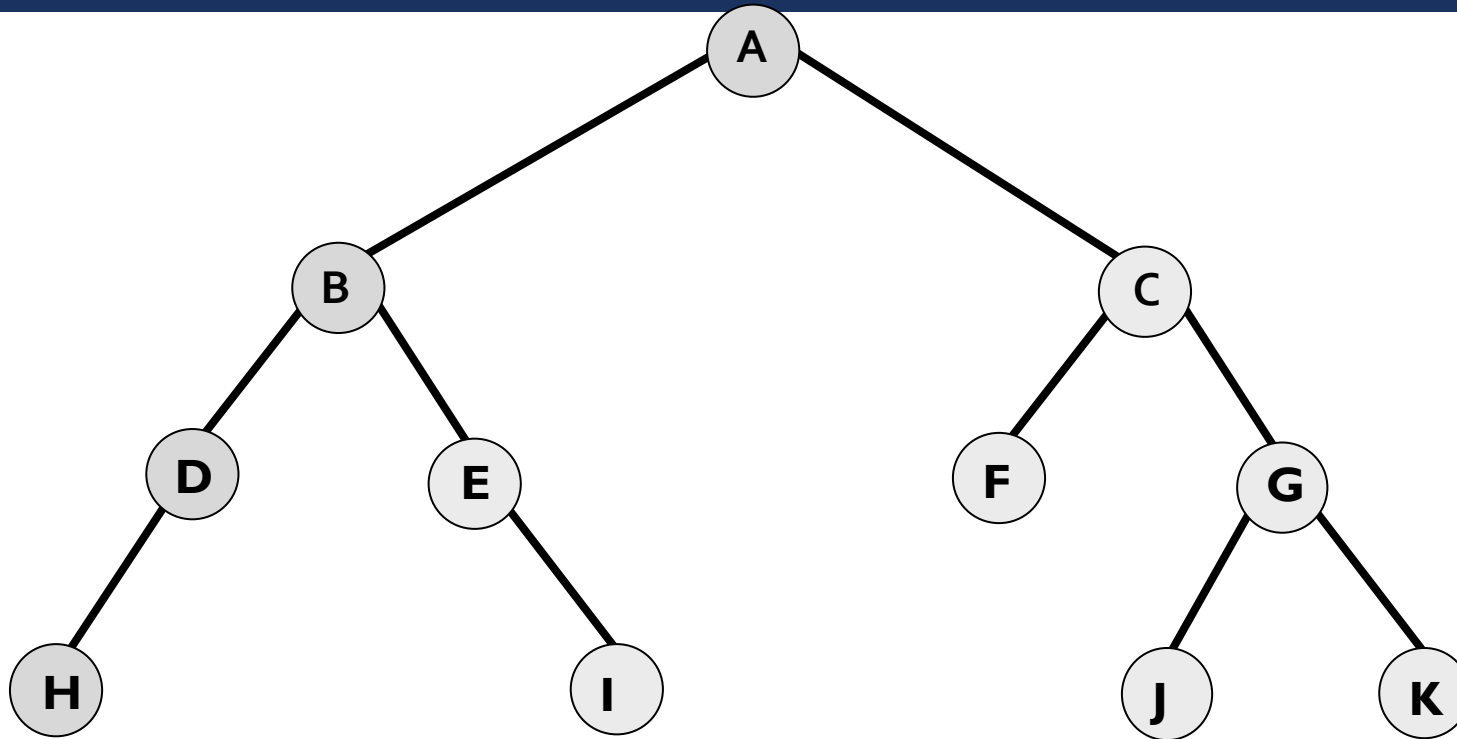


A

B

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH



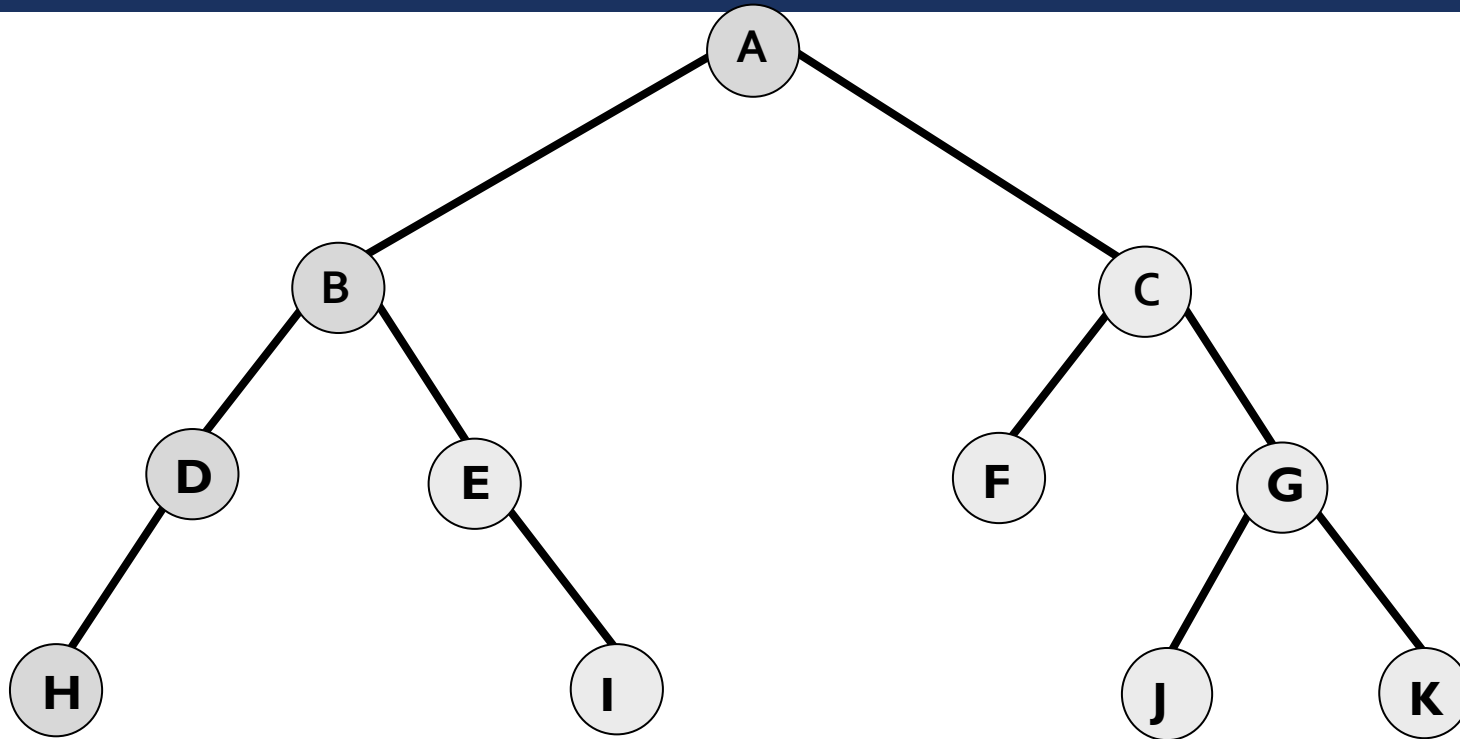
S



A B

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH



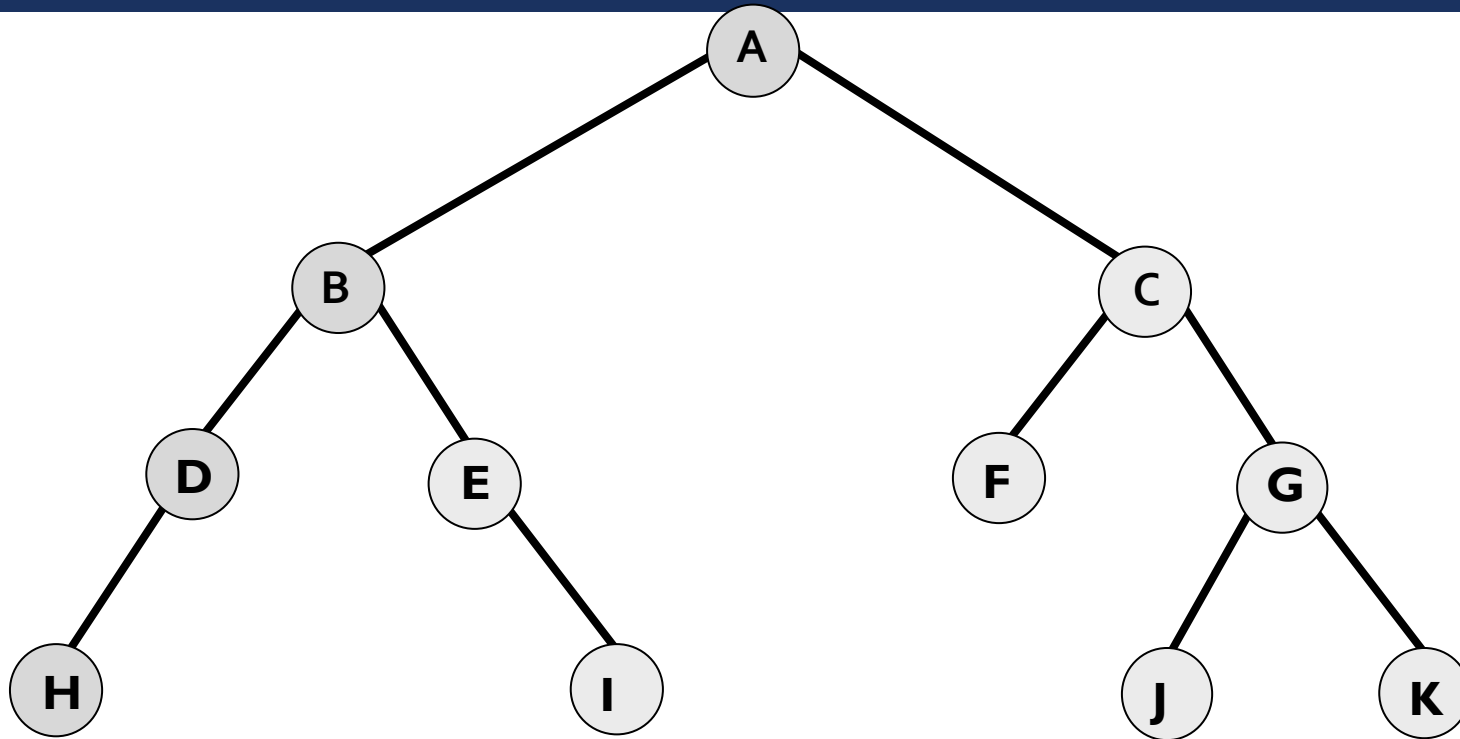
S



A B D

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

DEPTH-FIRST SEARCH



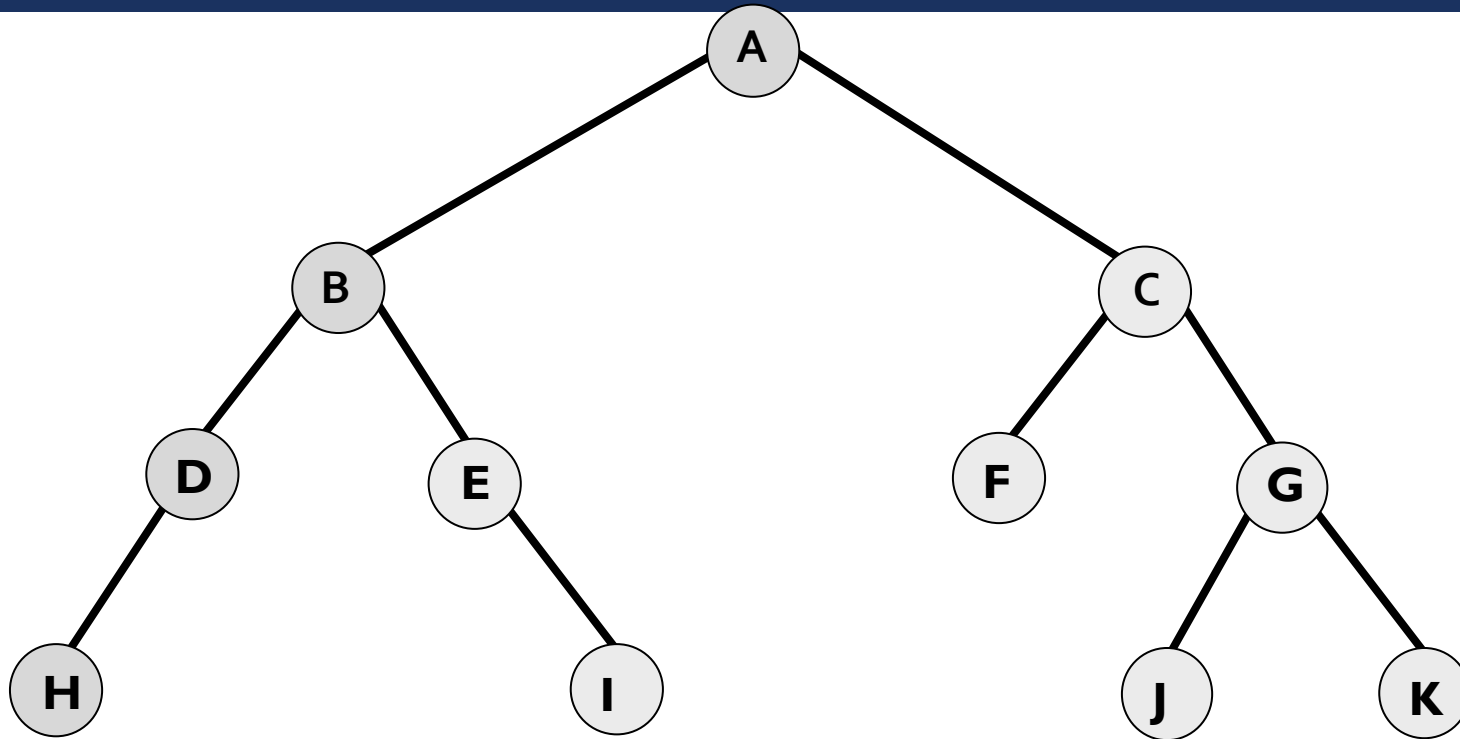
S



A B D

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

DEPTH-FIRST SEARCH



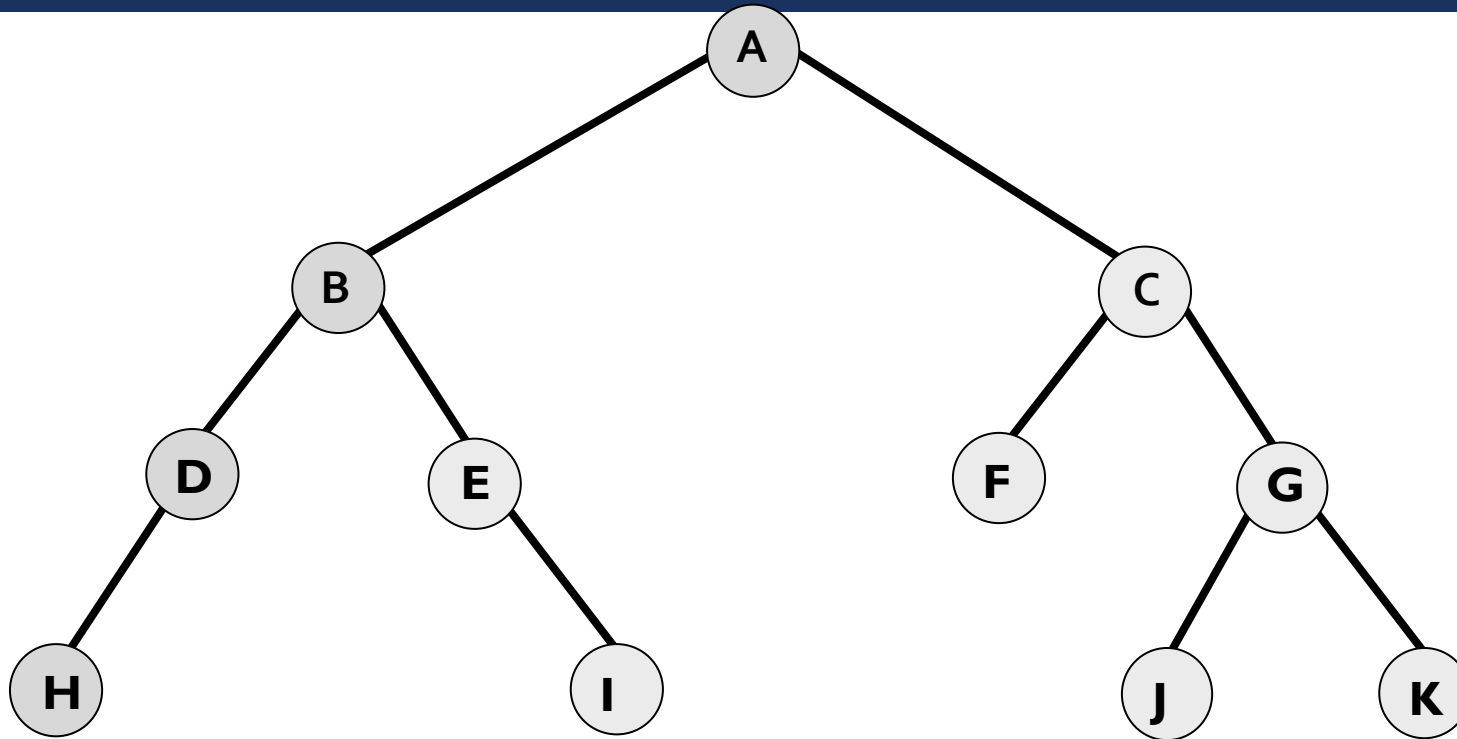
S



A B D H

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

DEPTH-FIRST SEARCH



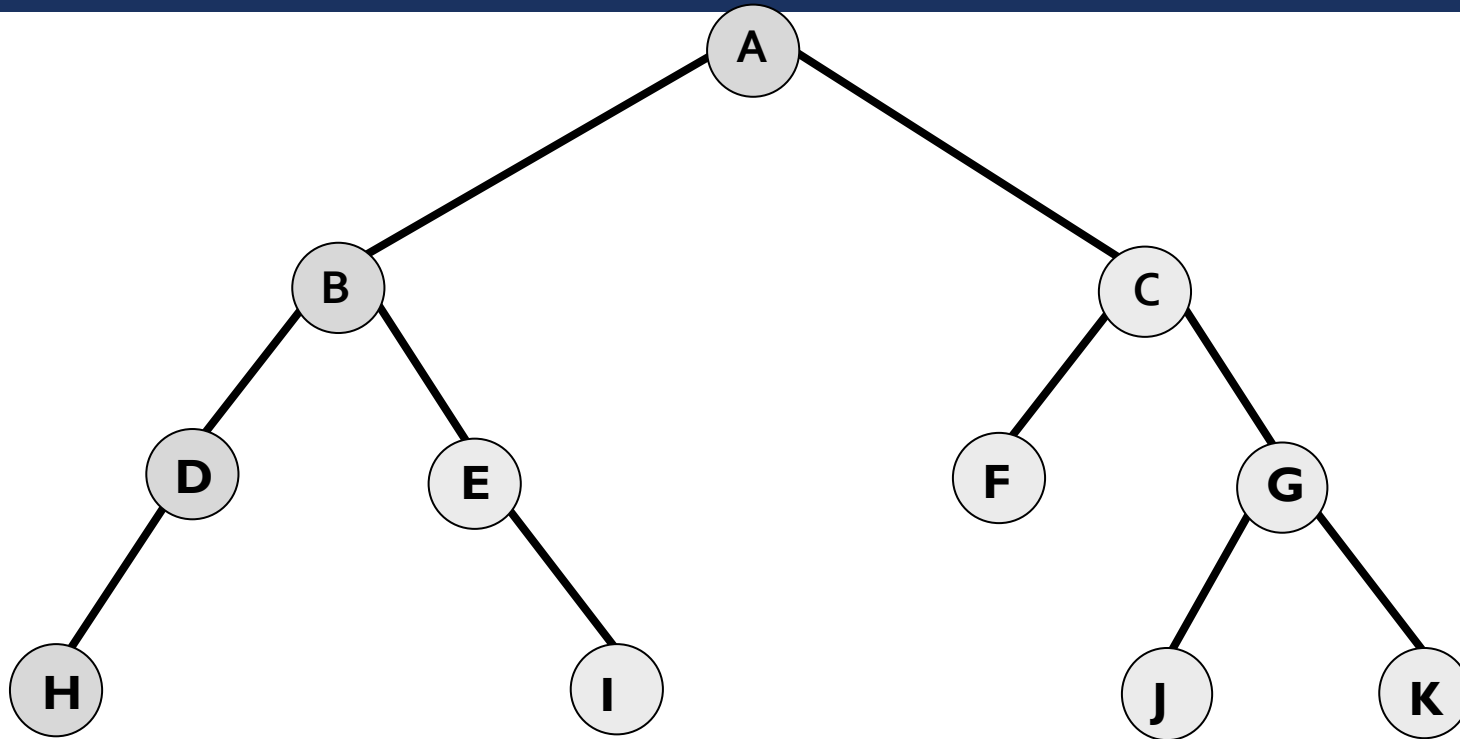
S



A B D H E

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH



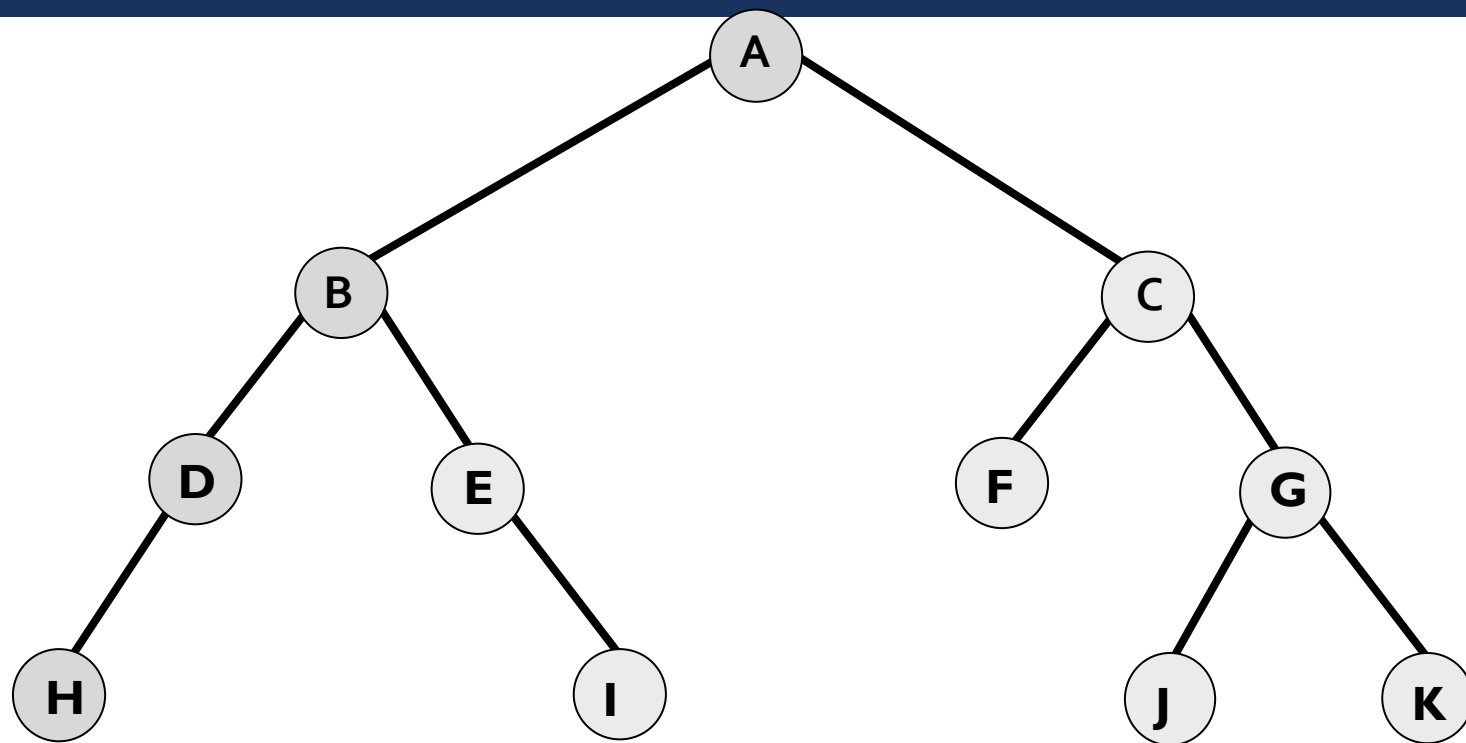
S



A B D H E

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

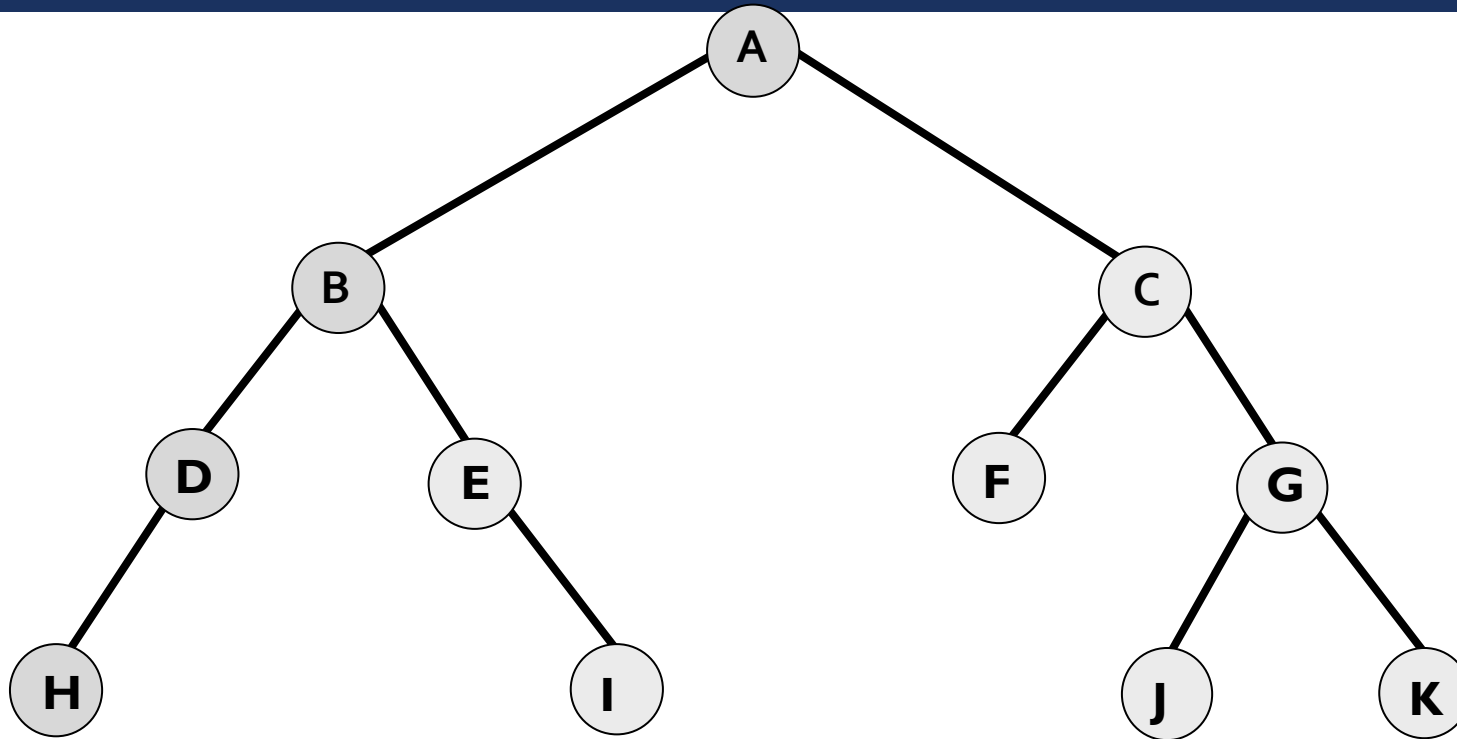
DEPTH-FIRST SEARCH



S	C									
	A	B	D	H	E	I				

NOD E	CHILDRE N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH



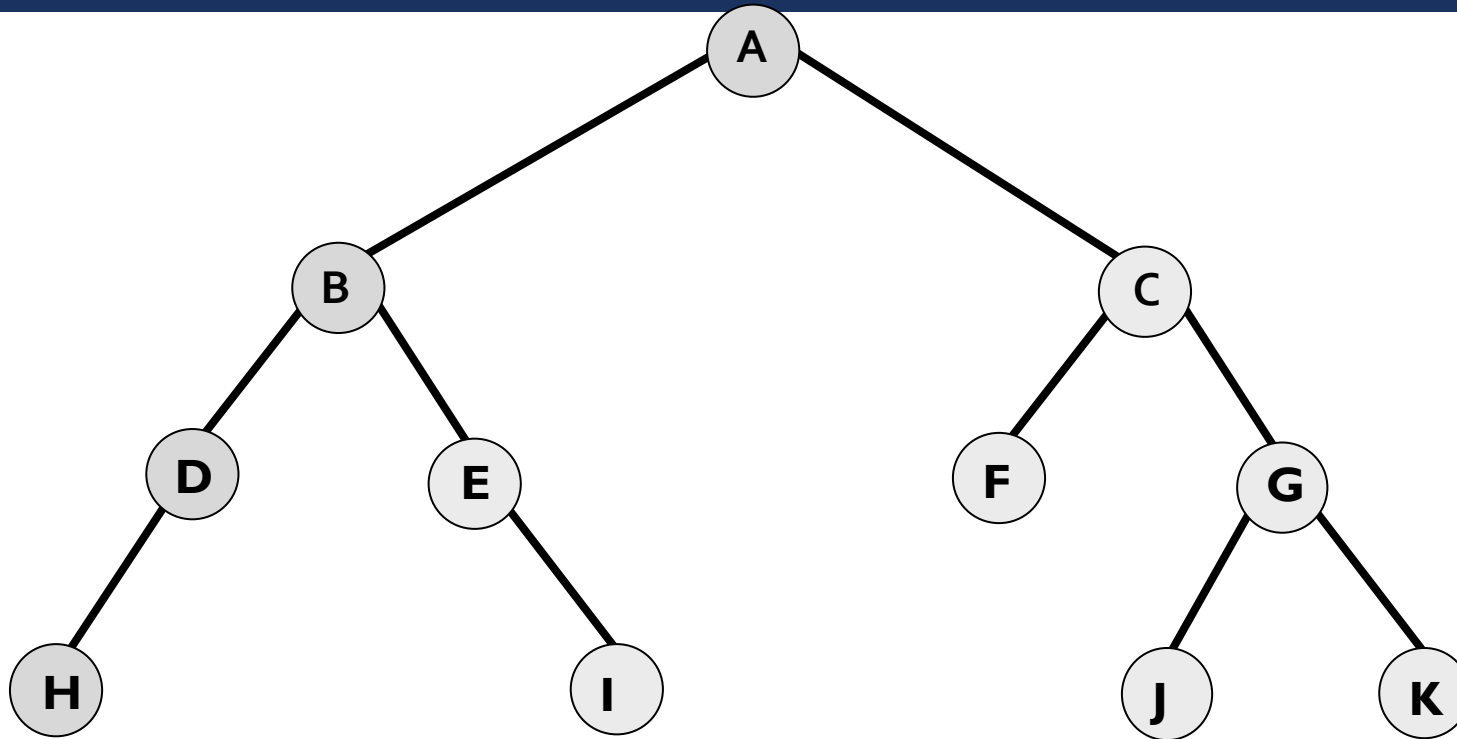
S



A B D H E I C

NOD E	CHILDRE N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

DEPTH-FIRST SEARCH



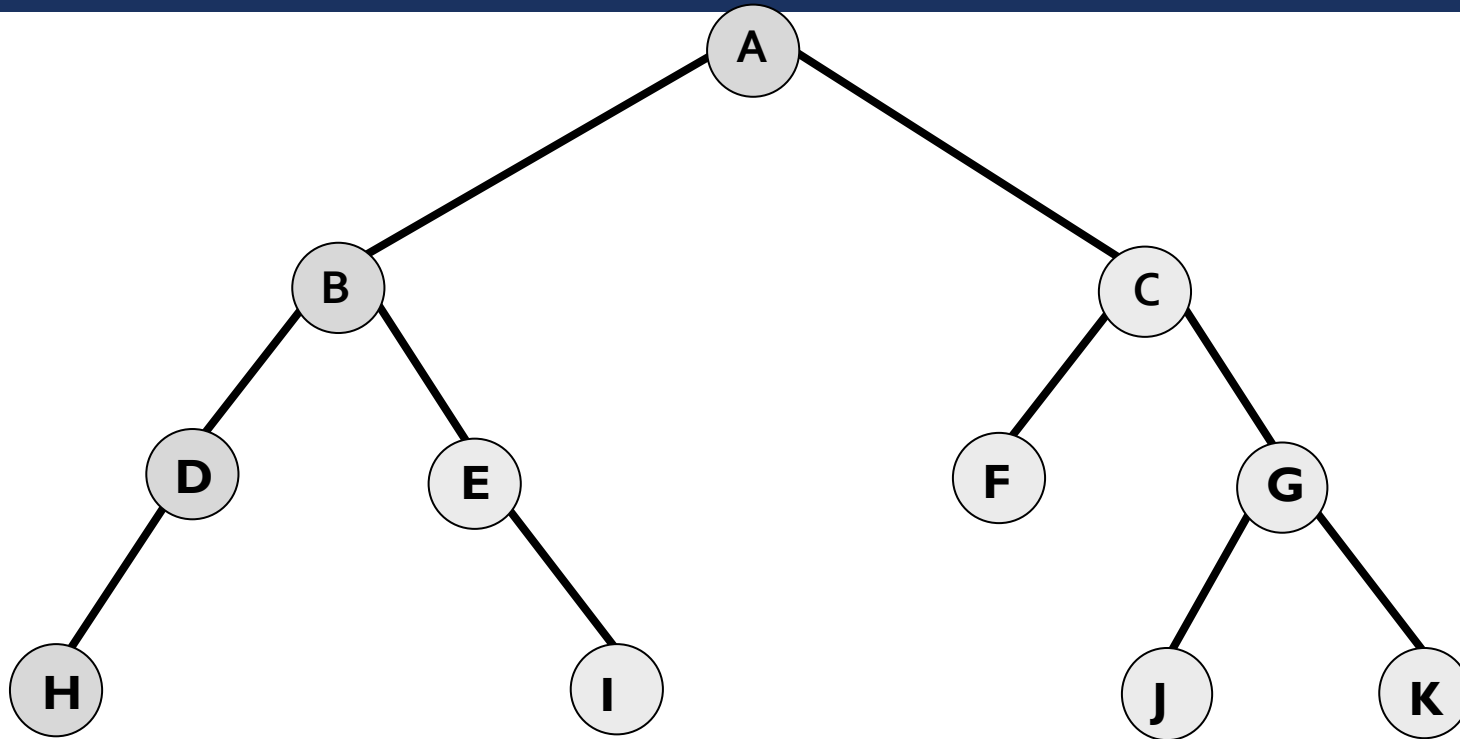
S



A B D H E I C

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

DEPTH-FIRST SEARCH



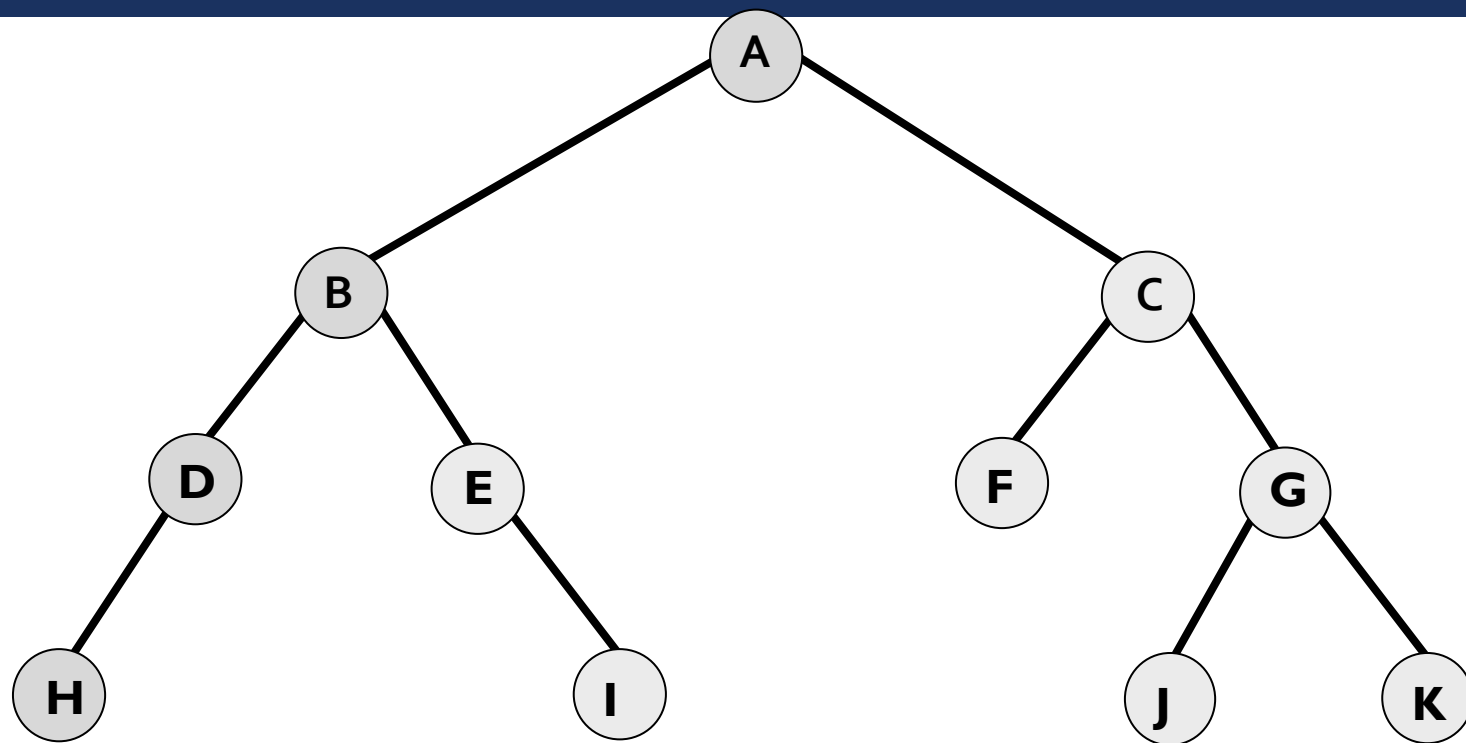
S



A B D H E I C F

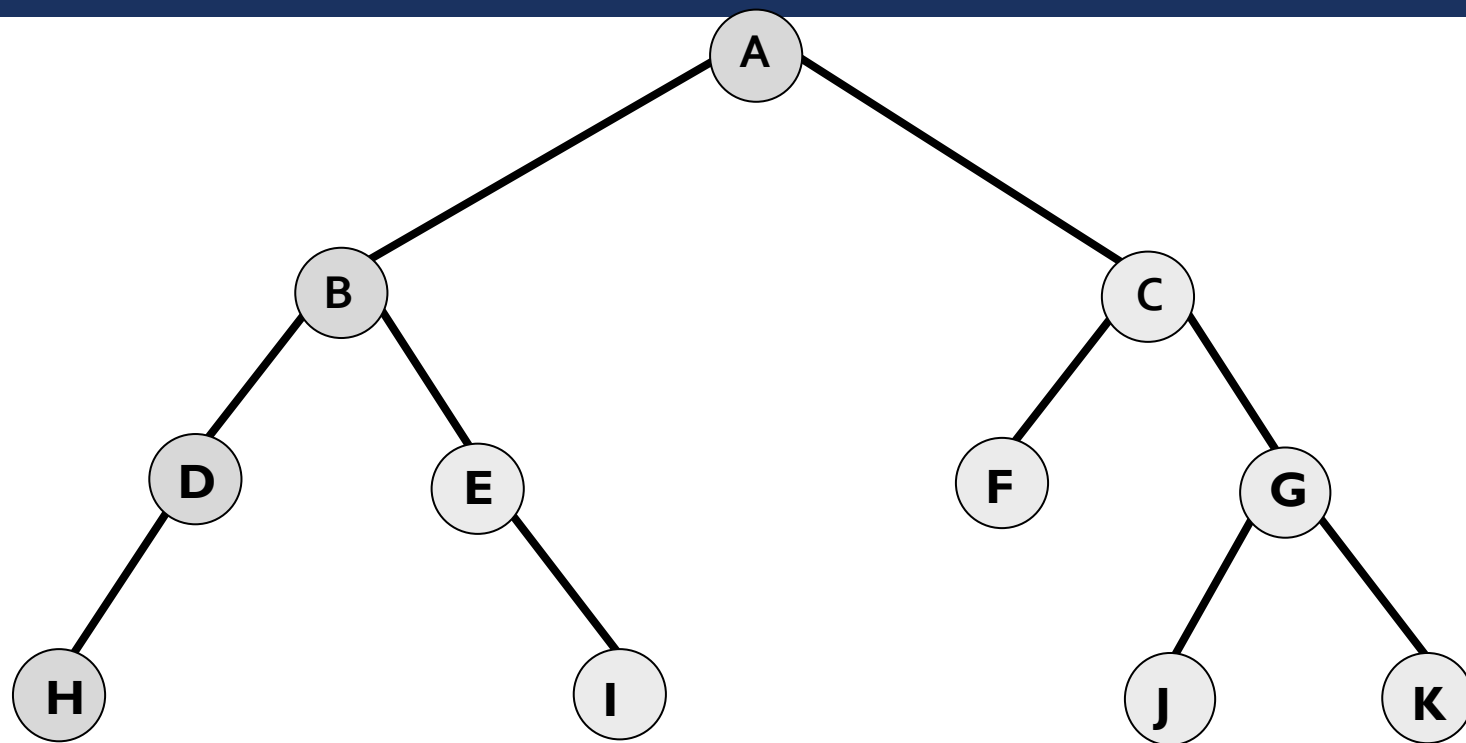
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH



NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

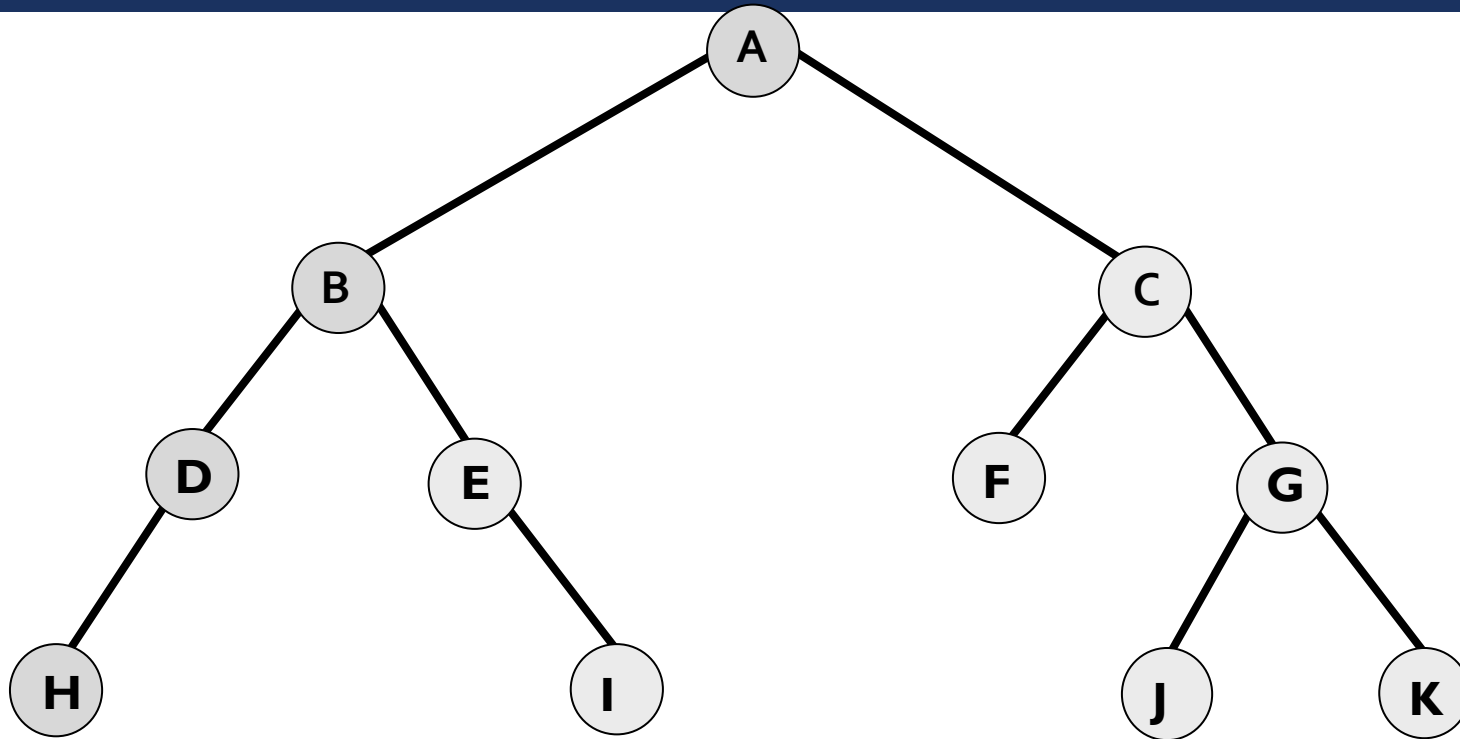
DEPTH-FIRST SEARCH



S	K	J								
	A	B	D	H	E	I	C	F	G	

NOD E	CHILDRE N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

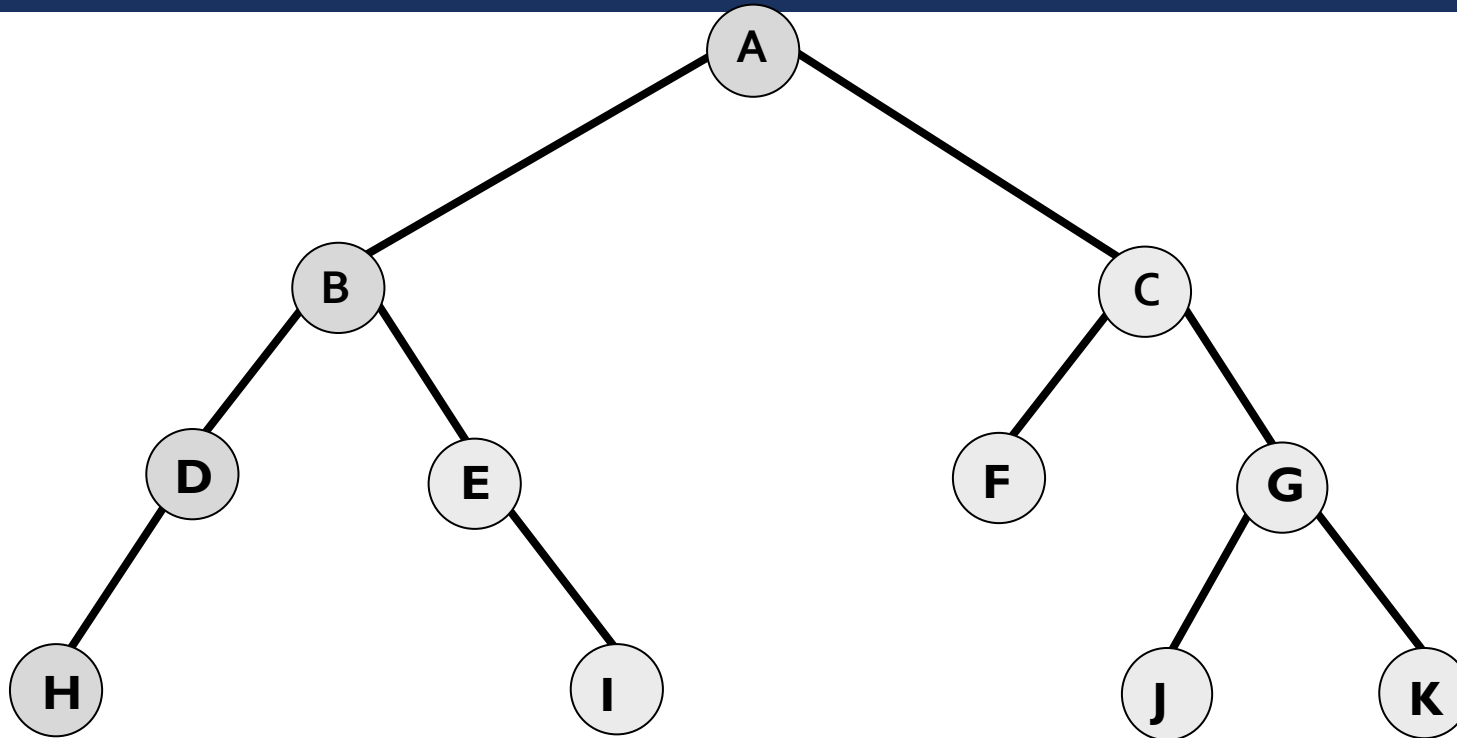
DEPTH-FIRST SEARCH



S	K									
	A	B	D	H	E	I	C	F	G	J

NOD E	CHILDRE N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DEPTH-FIRST SEARCH



S



A B D H E I C F G J K

NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

DFS: ALGORITHM AND TIME COMPLEXITY

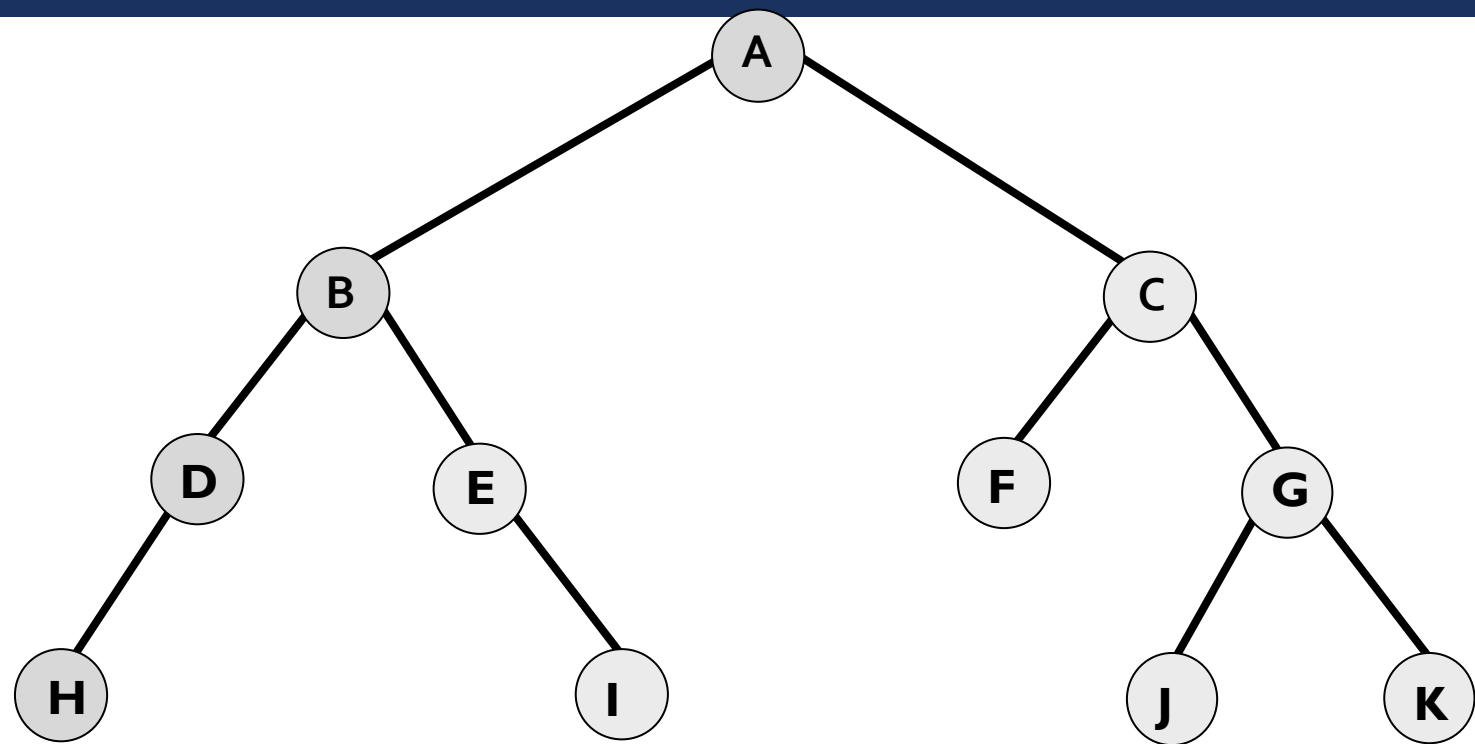
```
DFS(Root)
  Stack S
  S.push(Root)

  while S is not empty do
    v ← S.pop()
    Print v

    for each child w of v do
      S.push(w)
```

$$O(|E| + |V|) = O(n)$$

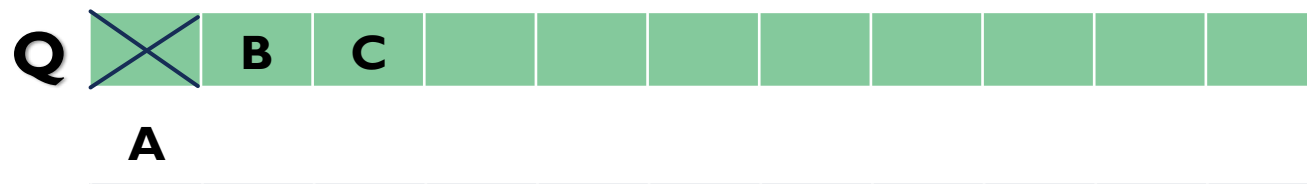
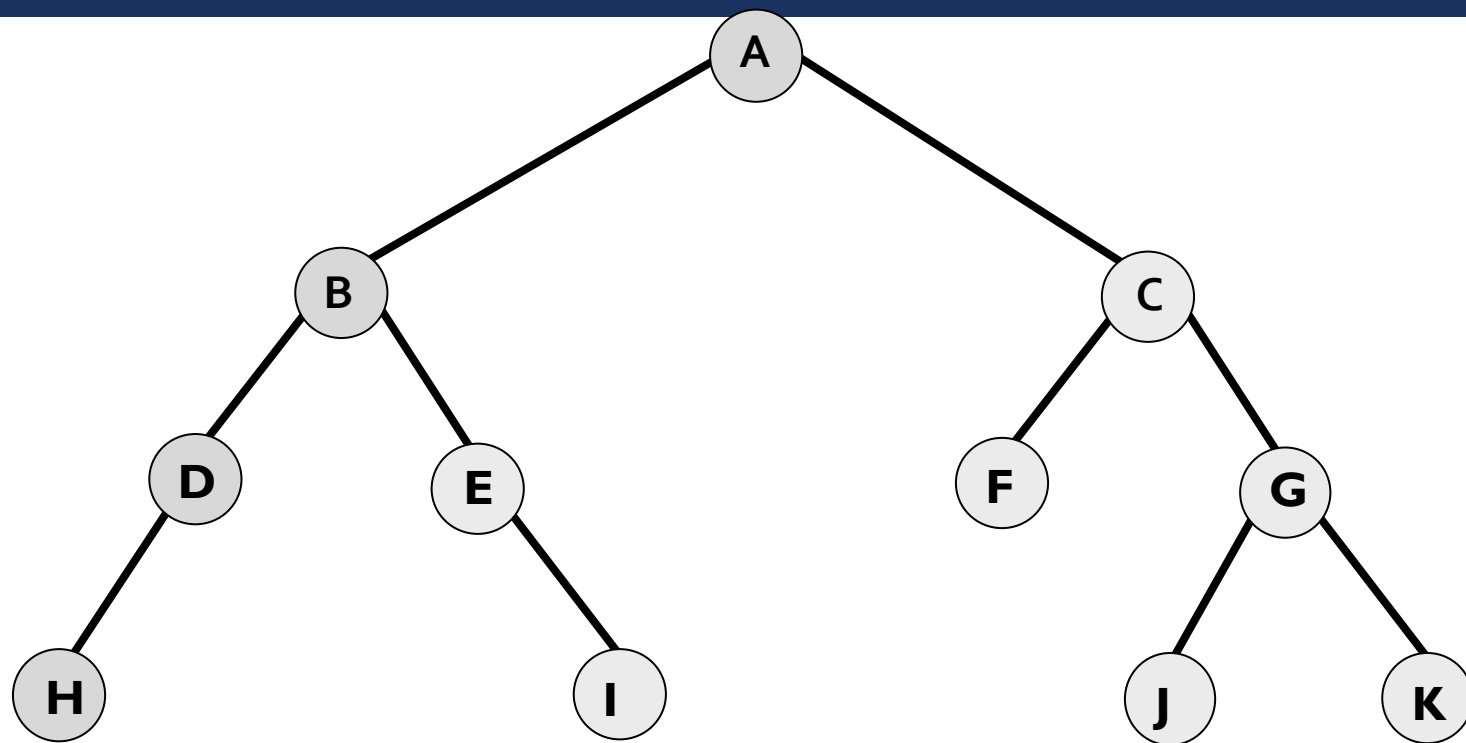
BREADTH-FIRST SEARCH



Q A

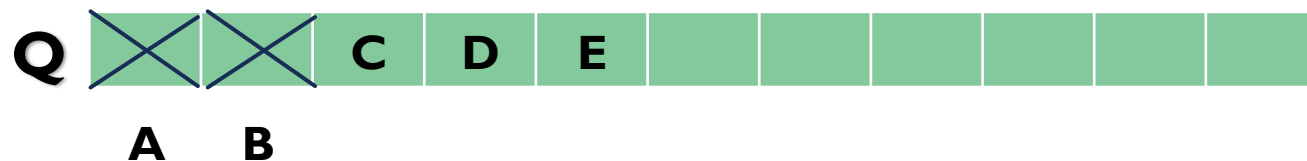
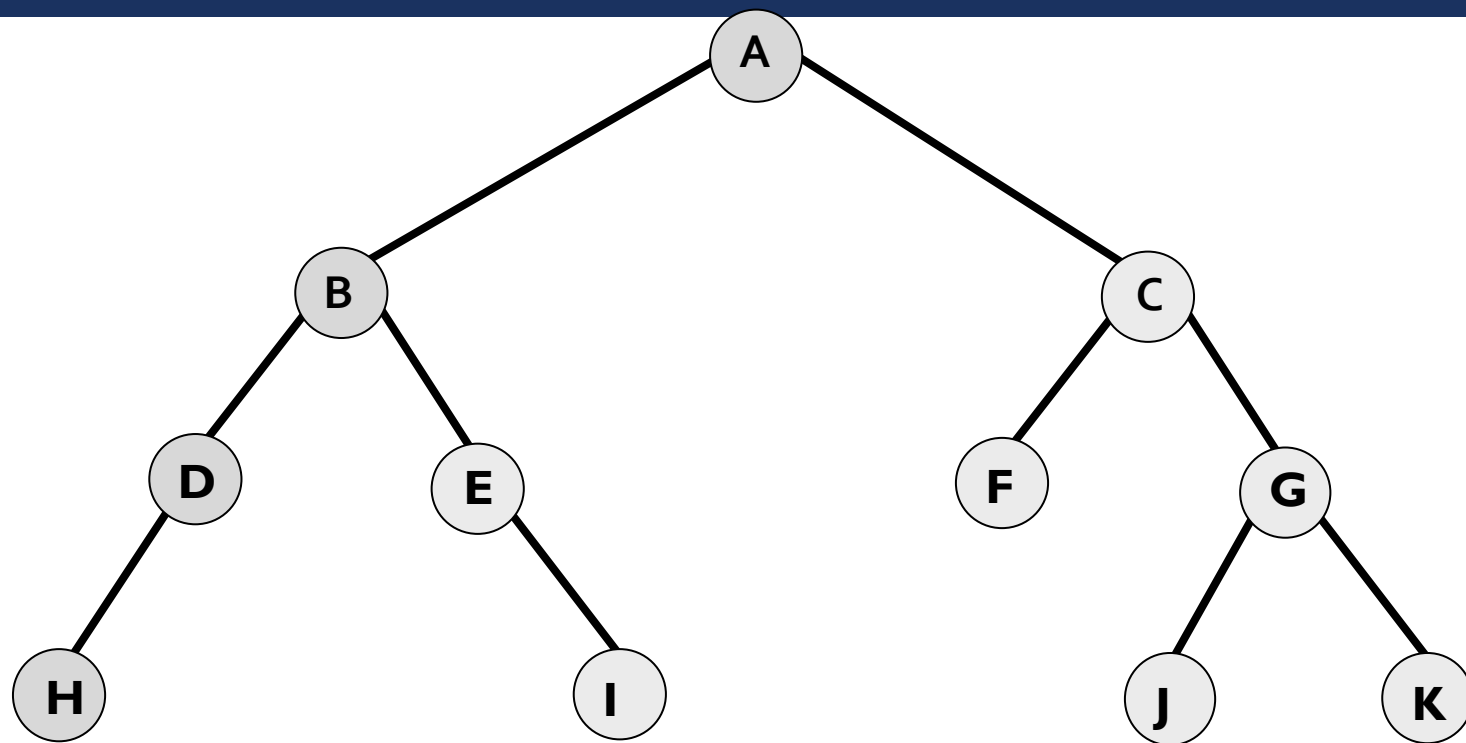
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



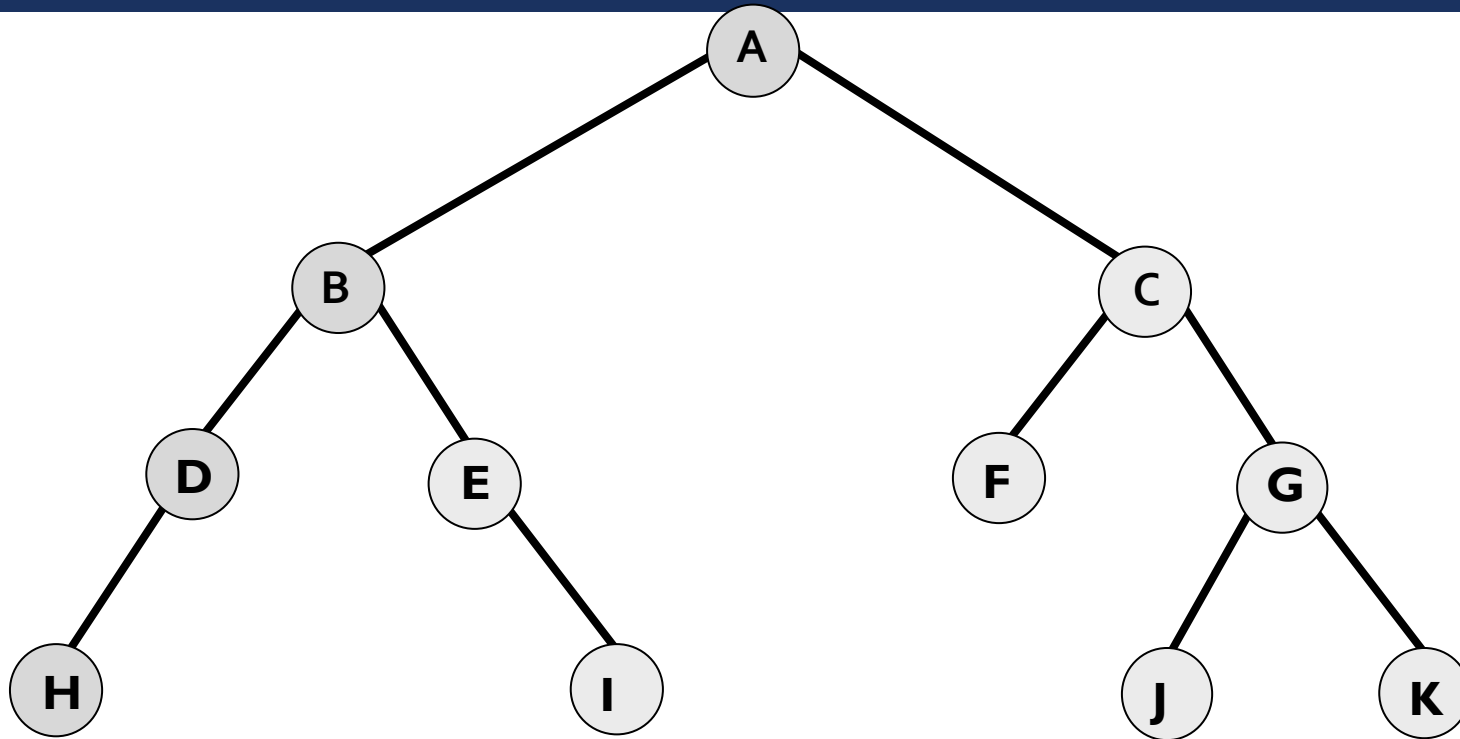
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



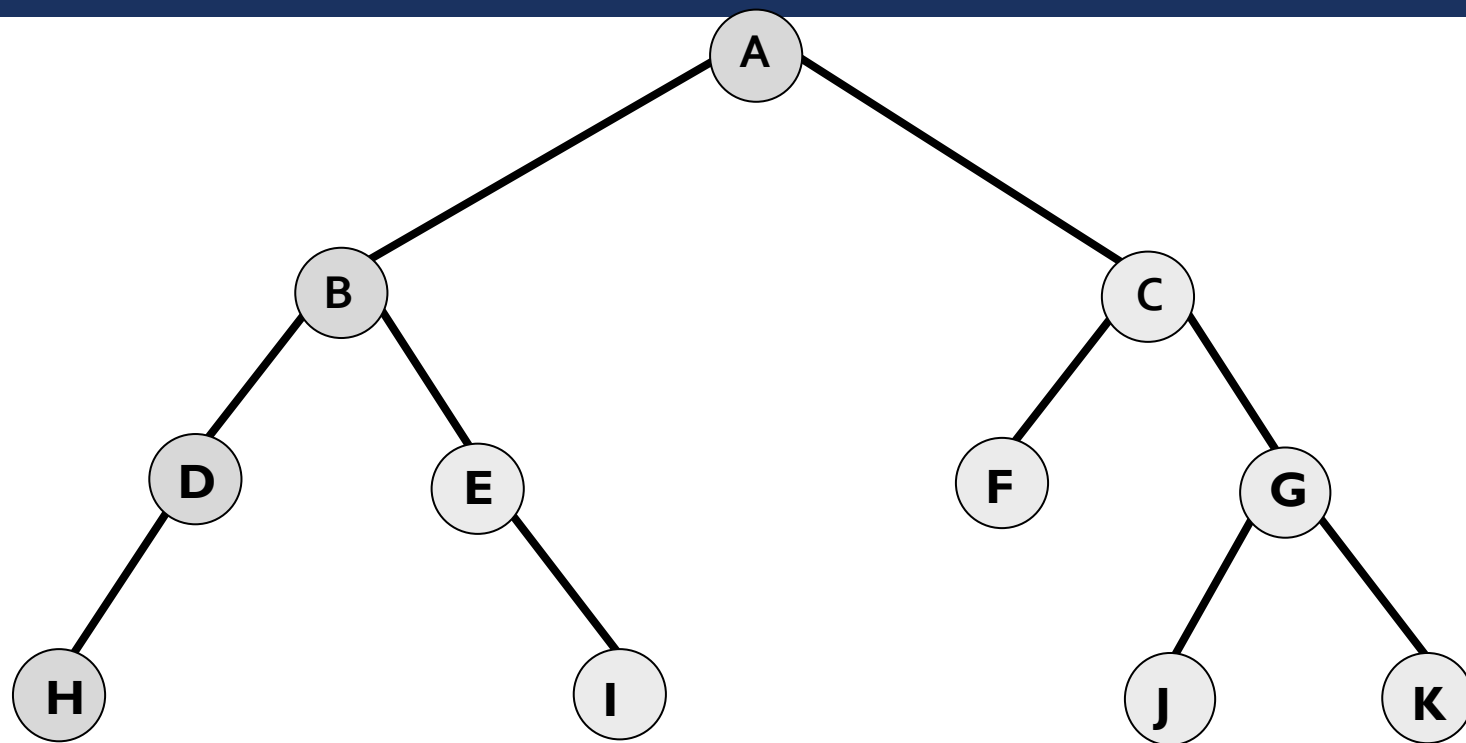
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

BREADTH-FIRST SEARCH



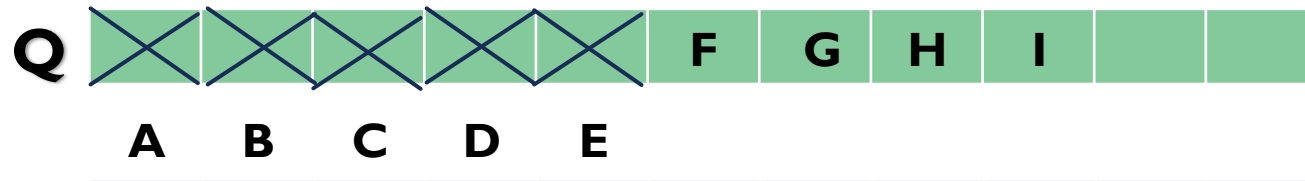
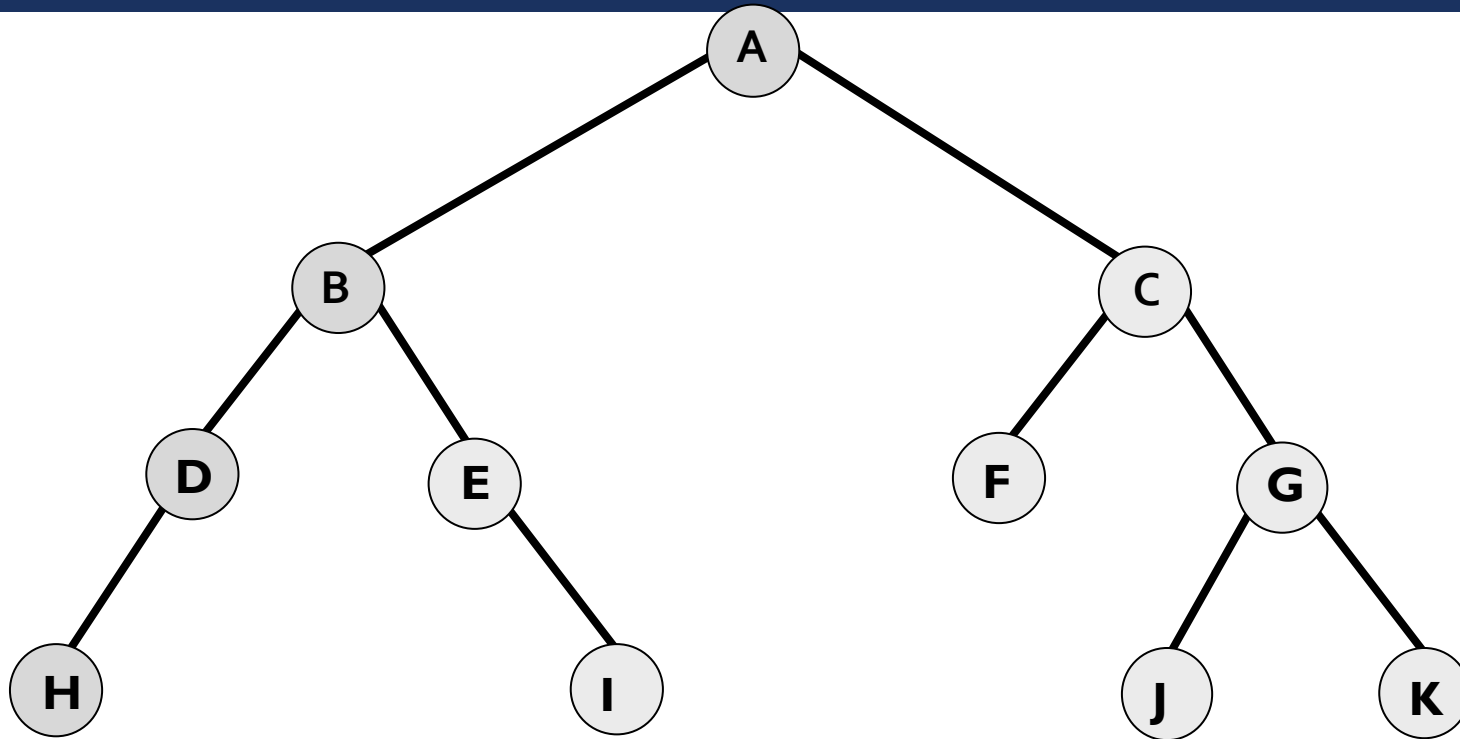
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

BREADTH-FIRST SEARCH



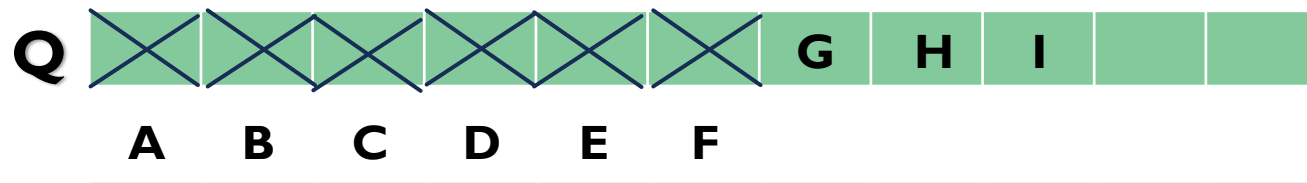
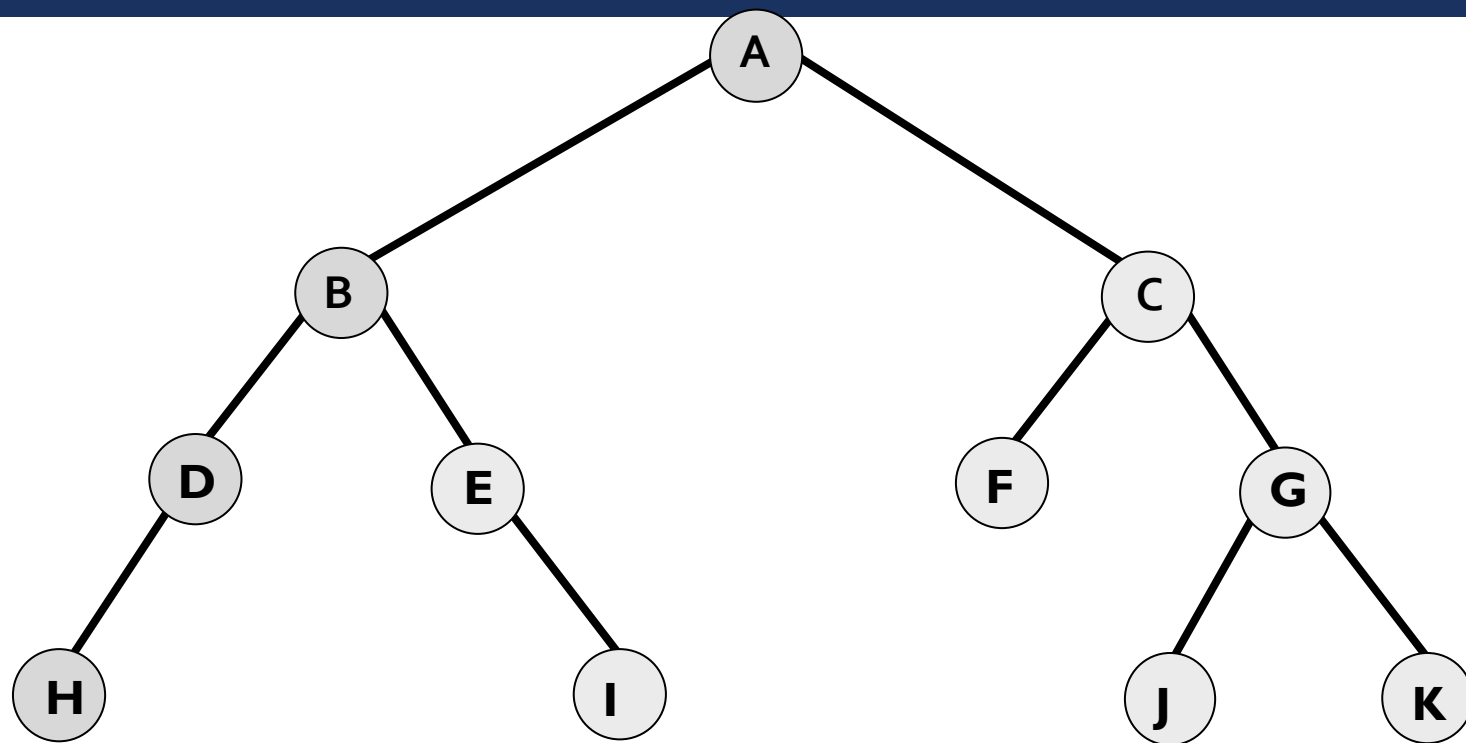
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



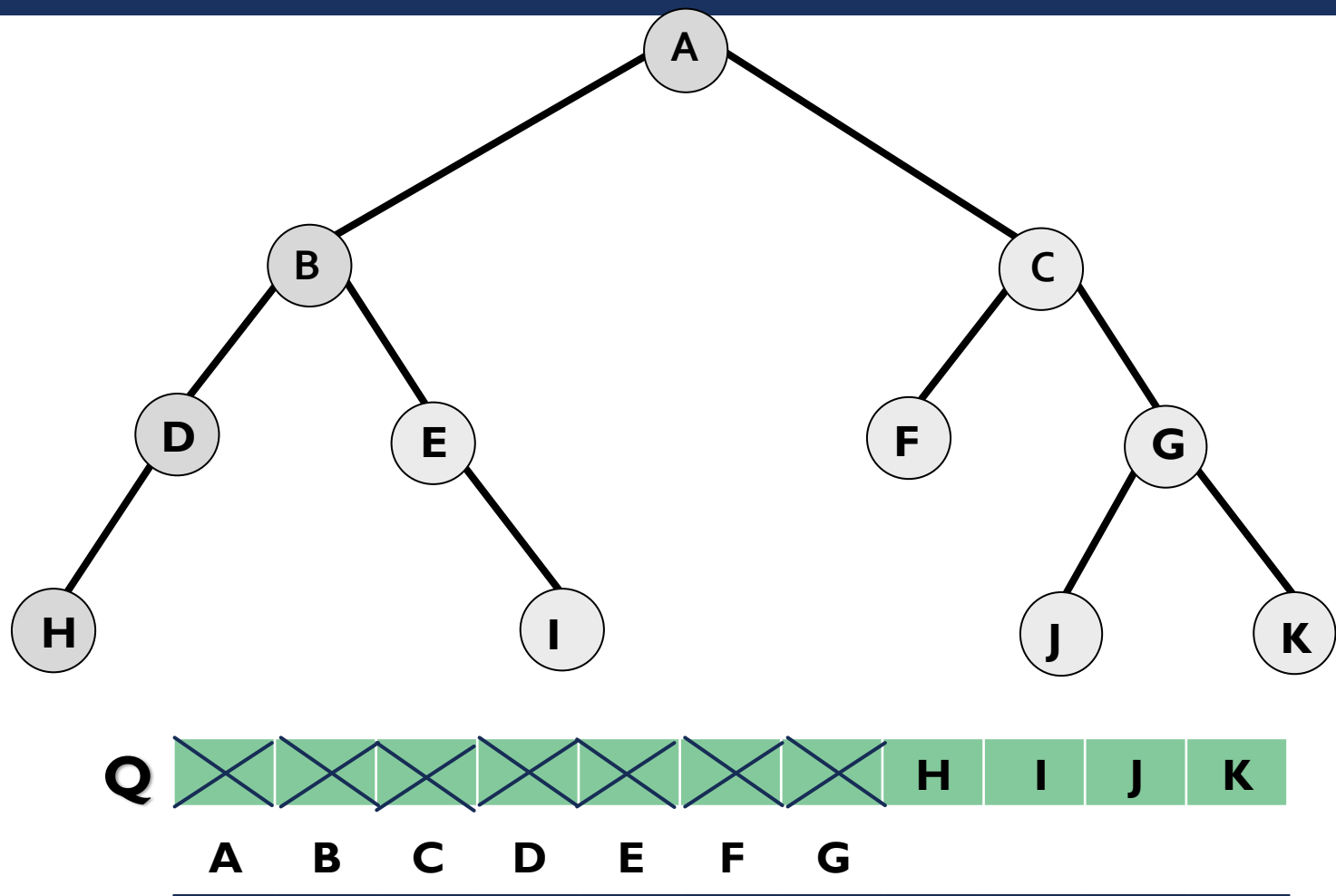
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-,-
G	J,K
H	-,-
I	-,-
J	-,-
K	-,-

BREADTH-FIRST SEARCH



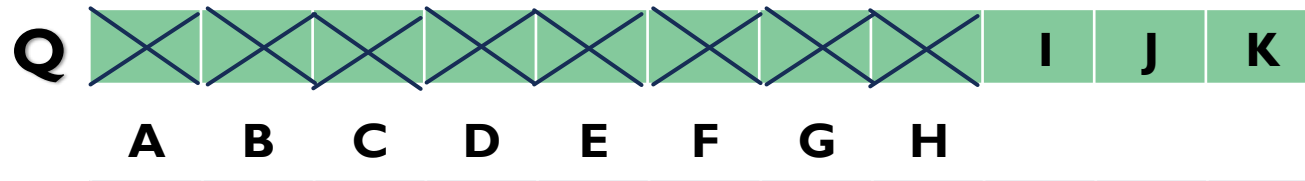
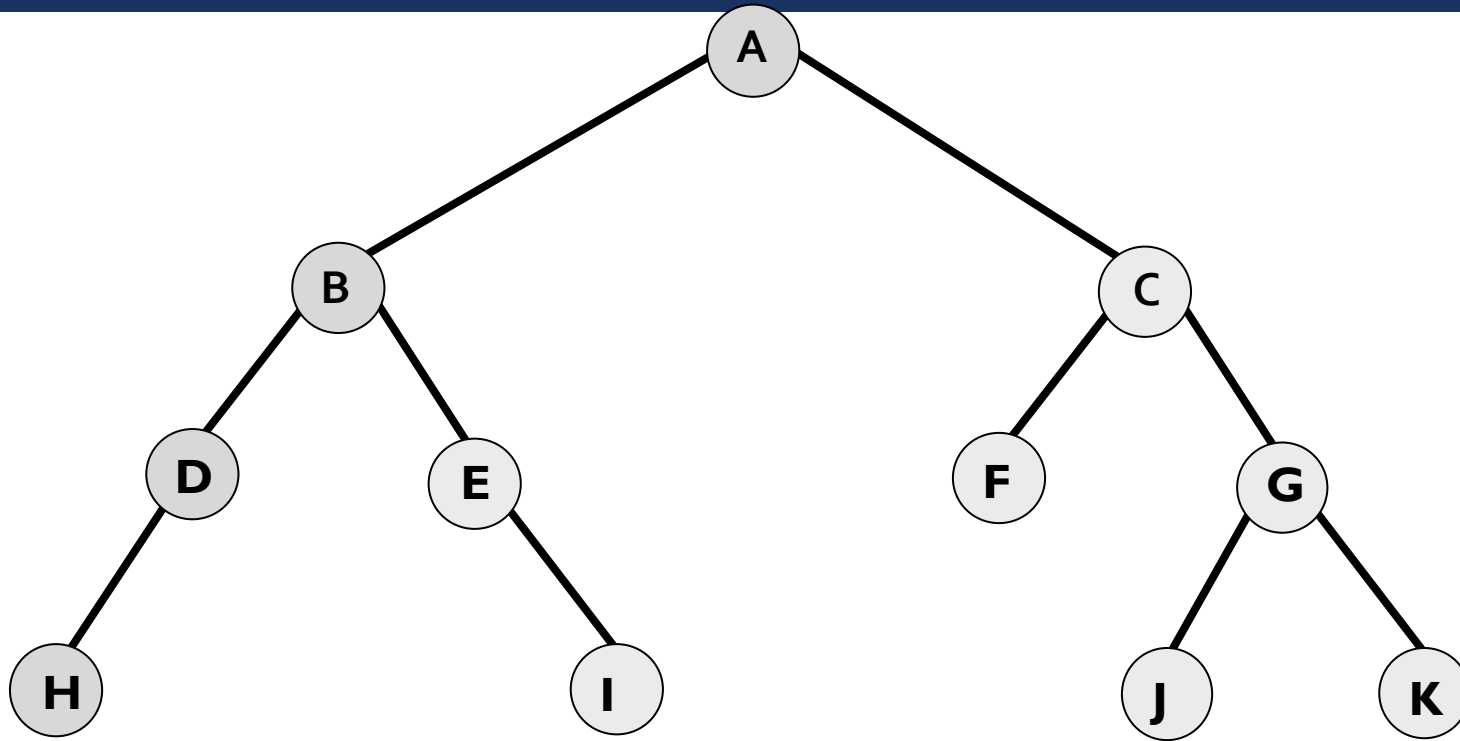
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



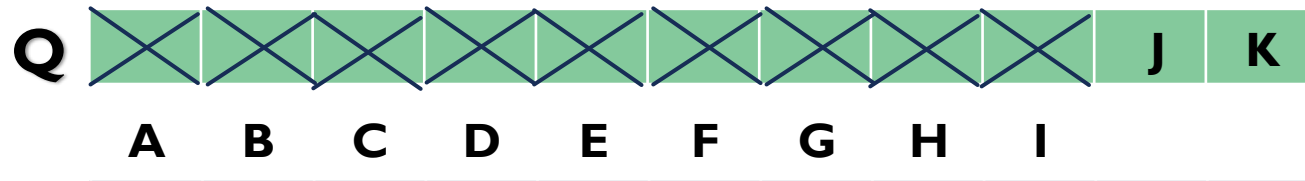
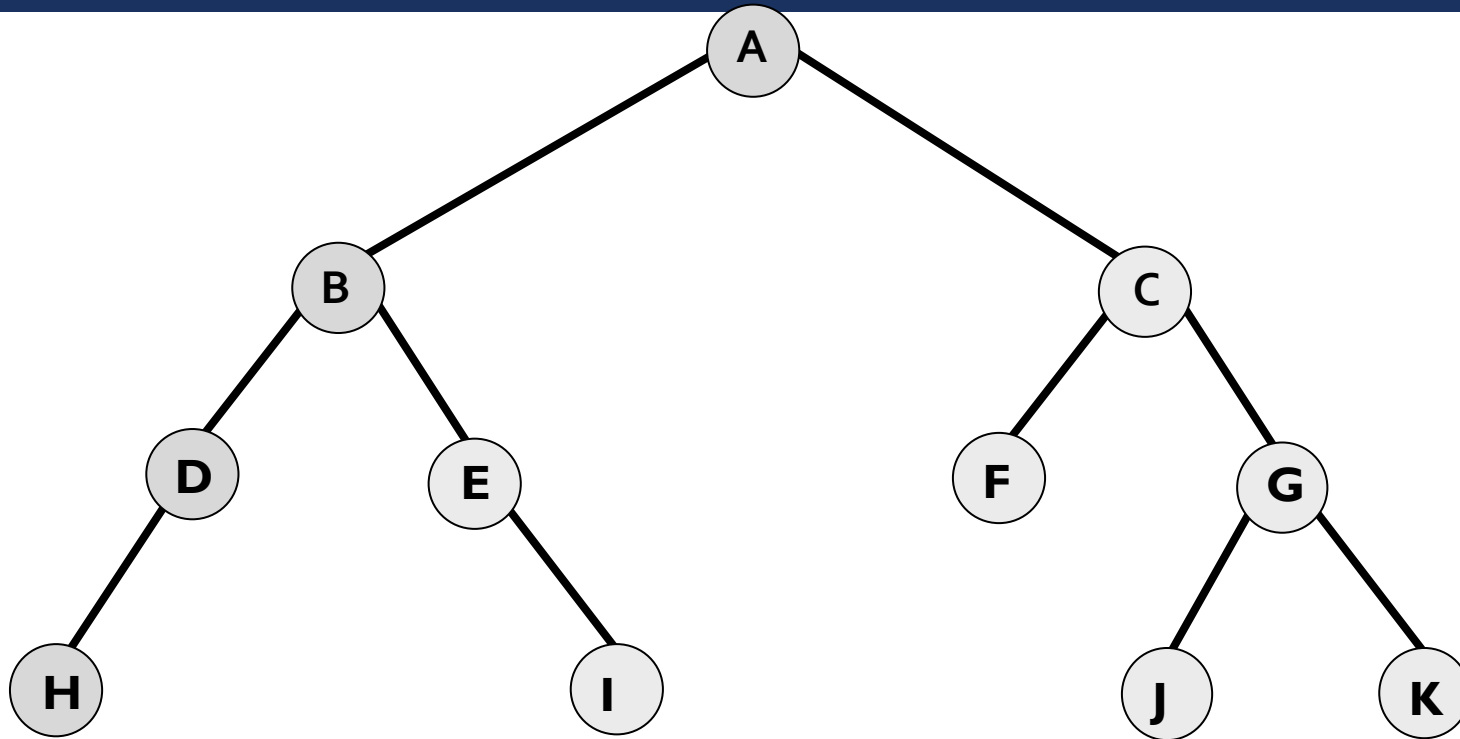
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



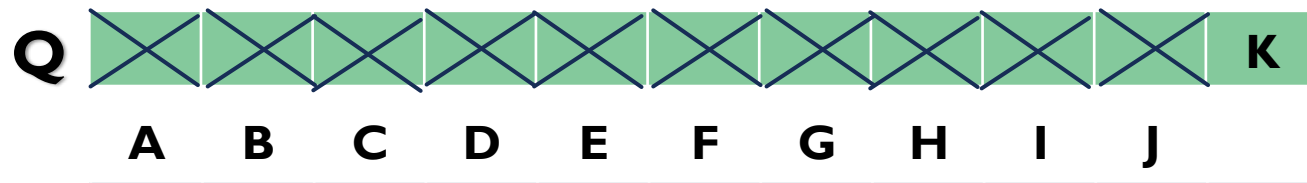
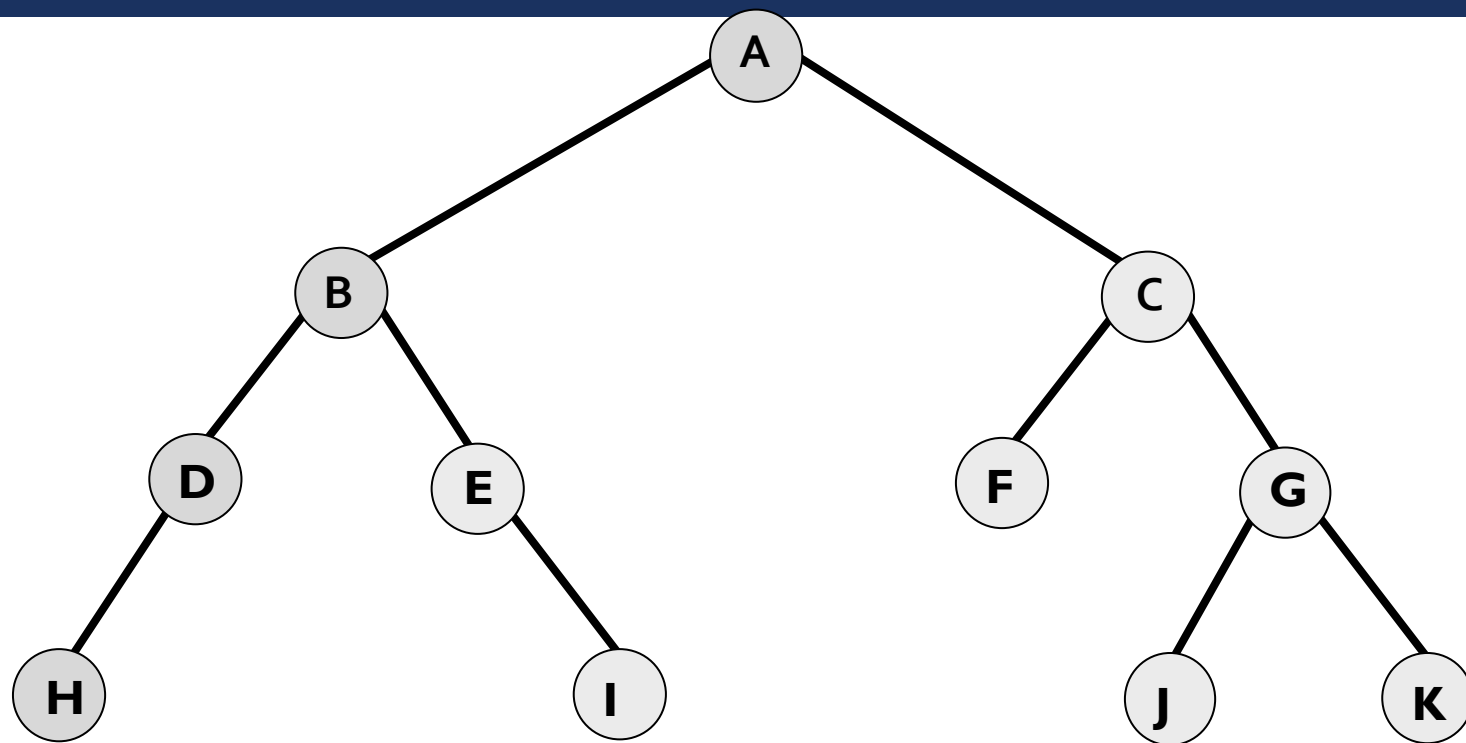
NOD E	CHILDRE N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



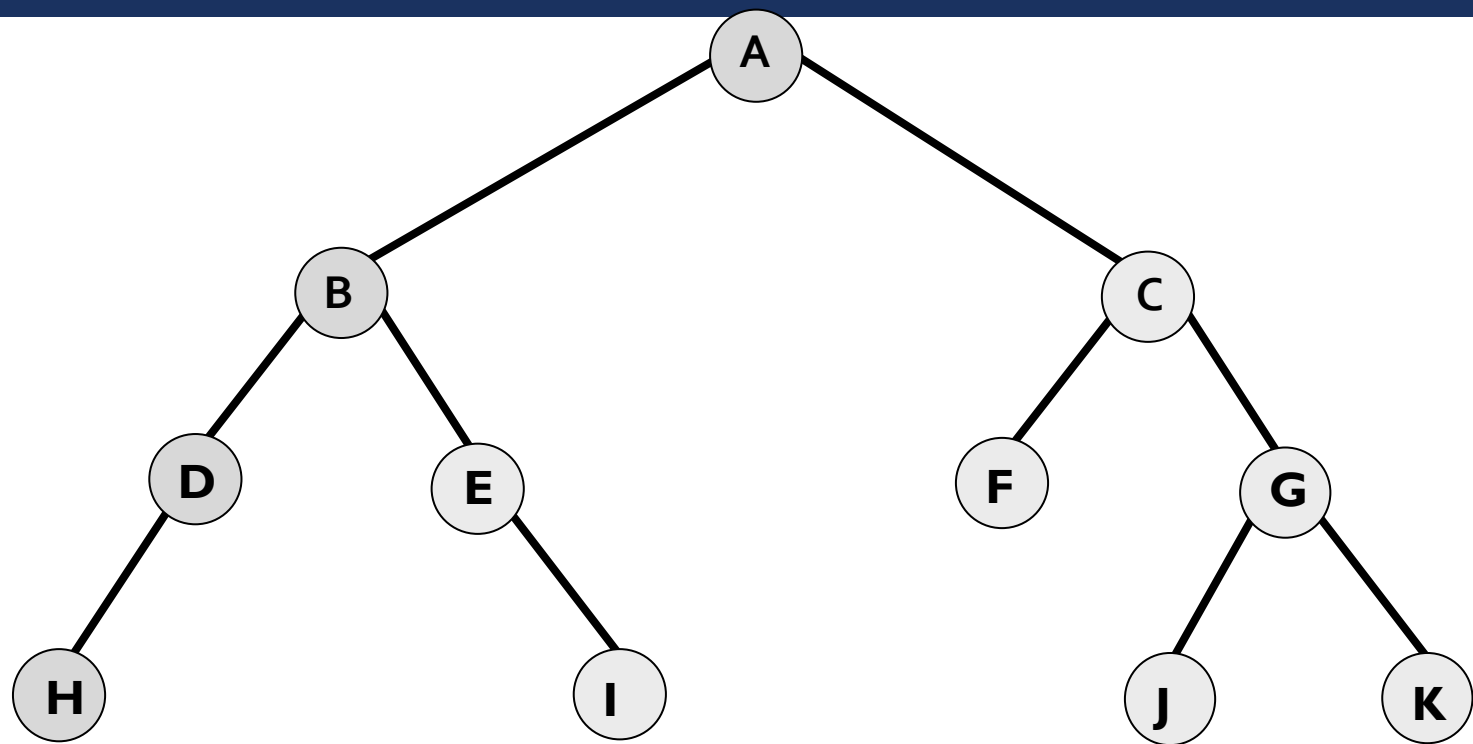
NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



NOD E	CHILDRE N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BREADTH-FIRST SEARCH



NOD E	CHILDR N
A	B,C
B	D,E
C	F,G
D	H,-
E	-,I
F	-, -
G	J,K
H	-, -
I	-, -
J	-, -
K	-, -

BFS:ALGORITHM AND TIME COMPLEXITY

```
BFS(Root)
    Queue Q
    Q.add(Root)

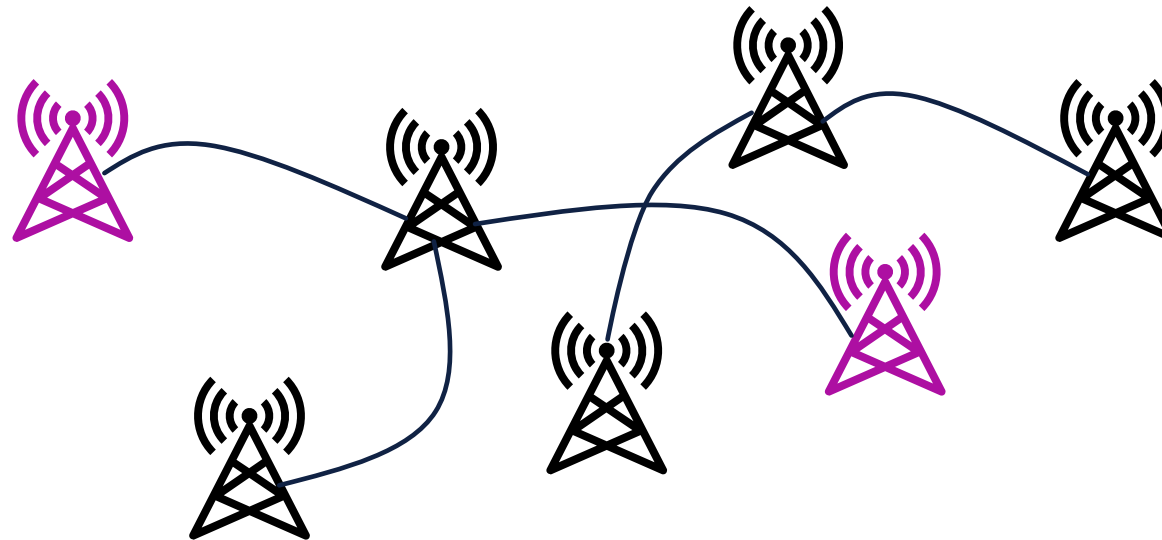
    while Q is not empty do
        v ← Q.delete()
        Print v

        for each child w of v do
            Q.add(w)
```

$$O(|E| + |V|) = O(n)$$

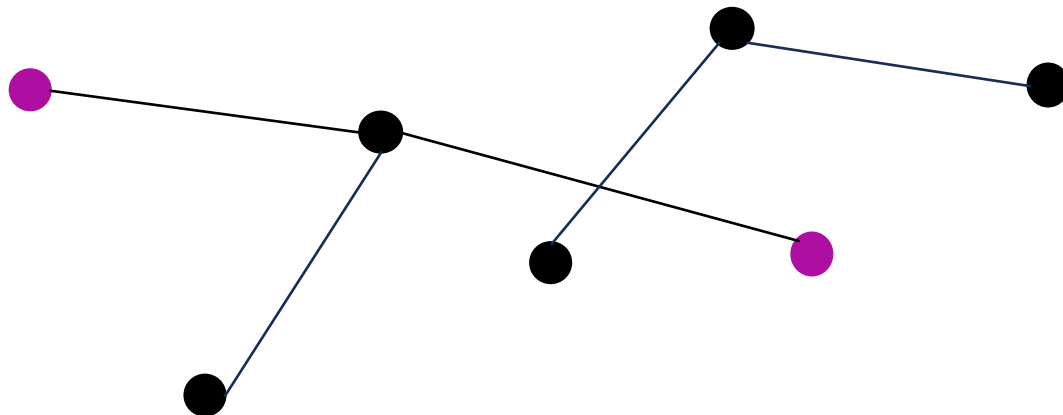
CLASS WORK

- You are managing a network of n communication towers in a large rural region. There are some **control centers**, each based at one of the towers. Your goal is to ensure that every tower in the region is connected to at least one control center. Obviously, the tower that has a control center is connected to its own control center.



CLASS WORK

- You are managing a network of n communication towers in a large rural region. There are some **control centers**, each based at one of the towers. Your goal is to ensure that every tower in the region is connected to at least one control center. Obviously, the tower that has a control center is connected to its own control center.



CLASS WORK

```
COVER(G, n, C):  
    # G: adjacency list of the graph with n nodes labeled 1..n  
    # C: set of control centers  
    visited ← empty set  
    for each c in C:  
        if c not in visited:  
            DFS(c, visited)  
    if size(visited) = n:  
        return TRUE  
    else:  
        return FALSE
```

```
DFS(u, visited):  
    visited.add(u)  
    for each v in G[u]:  
        if v not in visited:  
            DFS(v, visited)
```

ANY QUESTION??

HAVE A NICE DAY!